



Ferhat Abbas University Setif 1

Faculty of Computer Science

Linux Lateral Movement Detection with Graph Neural Networks

Master's Thesis

Candidates:

Houda NOUAS

Maroua ZIGHMI

Supervisor:

Prof. Lyazid TOUMI

Academic Year: 2025–2026

They said it would be hard.

They were right.

This page is for everyone who made it possible anyway.

To myself —

I want to thank me. For believing in me
when no one else was in the room.

For getting up one more time
every single time I fell.

This did not happen to me. I made it happen.

To my parents —

You gave me life twice:
once at birth, and once every time
you chose to sacrifice so I could go further
than you ever had the chance to.
No words in any thesis could repay that.

To my brother and sisters —

Home is not a place. It is you.
Thank you for being my safe landing
every time the world felt too loud.

To my thesis partner —

We started with a blank page
and built something worth reading.
Thank you for showing up, every single day,
with your mind sharp and your patience sharper.

To my friends —

You never asked to read my chapters.
You just made sure I remembered
there was a life worth living outside of them.
That was enough. That was everything.

And to everyone who believed in me —

Your belief was not passive.
It was a force. It pushed me forward
on days when I had nothing left to push with.
I carried every one of you to the finish line.

written by: NOUAS Houda

الإهداء

بسم الله الرحمن الرحيم، والصلاة والسلام على سيدنا ونبينا محمد صلى الله عليه وسلم.

بعد تعبٍ ومشقةٍ لسنواتٍ عدة، ها أنا اليوم أقف على عتبةٍ تخرجني رافعةً رأسي بكل فخر. فالحمد لله الذي نظن به خيرًا فيكرمننا بأفضل مما ظننا به، وما توفيقي إلا بالله. لم تكن مجهوداتي وحدها كافية، بل كان فضلك وكرمك يا الله حاضرًا في كل خطوة من هذا الطريق.

وبكل حب أهدي ثمرة نجاحي وتخرجي:

إلى من علمني أن الدنيا كفاح، وأن سلاحها العلم والمعرفة، سندي وقوتي وملذي بعد الله، والدي العزيز.

إلى من كان دعاؤها سر نجاحي، وكان دعمها مصدر قوتي، أُمِّي الغالية.

إلى من شد الله بهم عضدي وكانوا لي خير سند عند ضعفي، أولئك الذين منحوا للحب والأخوة معنىً أجمل في حياتي، إخوتي الأعزاء: أميمة، سندس، أيوب، وتسليم.

إلى جدتي الغالية، أدامها الله تاجًا على رؤوسنا، ورزقها الصحة والعافية وطول العمر.

إلى من طالما حلم بالافتخار بي ومرافقتي في نجاحاتي، ورغم غيابك عنا اليوم فإن حبك لا يغيب وأثرك لا يُنسى، رحمك الله وأسكنك فسيح جناته، جدي الغالي.

إلى كل من ساندني، وفرح لفرحي، وتمنى لي النجاح والتوفيق دائمًا، جزاكم الله خير الجزاء، ورزقكم الله هذا الفرح مع أبنائكم، أحوالي الأعزاء.

إلى رفقاء المشوار، شكرًا لدمكم ومشارككم تلك الأيام بحلوها ومرها، ستبقى ذكرياتها محفورة في قلبي وذاكرتي ما حبيت.

أما في النهاية، فإهدائي لنفسي...

إلى نفسي التي لم تستسلم، وتحملت الصعاب، وواصلت الطريق دون الالتفات إلى الوراء. سقطت فقامت، ولملمت شتاتها، ثم صنعت من نفسها نسخة أقوى وأصلب. حولت الخوف إلى شجاعة، واليأس إلى إصرار، ولم تنتظر العون إلا من الله.

ها أنتِ اليوم تجنين ثمرة تعبك، وتوقعين أول انتصاراتك، وما هذا إلا بداية طريق مليء بالنجاحات بإذن الله.

نسأل الله التوفيق والسداد، والحمد لله الذي هدانا لهذا وما كنا لنهتدي لولا أن هدانا الله.

زيغمي مروة

Acknowledgements

First and foremost, I would like to express my sincere gratitude to my supervisor, **Prof TOUMI Lyazid**, for their invaluable guidance, continuous support, and constructive feedback throughout the course of this research. Their expertise and dedication have been instrumental in shaping this work.

I would also like to extend my appreciation to the members of the examination committee for their time, their careful reading of this manuscript, and their insightful remarks.

I wish to express my heartfelt thanks to all the professors of the **Computer Science Department** who contributed to our training throughout this academic curriculum. Their teaching, knowledge, and commitment to excellence in the field of **Cybersecurity** have provided us with the theoretical foundations and practical skills that made this work possible.

My gratitude goes as well to the faculty and staff of **Computer Science Department, Ferhat Abbas University Setif 1**, for providing the academic environment and resources that made this research possible.

I am equally grateful to my thesis partner for the fruitful collaboration, the shared efforts, and the mutual support that sustained this work from its first draft to its final form.

Finally, I wish to express my deepest gratitude to my family, whose unwavering encouragement and sacrifices have been a constant source of strength and motivation throughout my academic journey.

Abstract

To identify lateral movement during cyber-attacks, we consider the problem of detecting malicious propagation activities within enterprise networks. Adversaries often use lateral movement to pivot from an initially compromised host toward additional high-value resources. Detecting such behavior is challenging due to the volume of streaming network traffic and the continuously evolving interactions between systems. In this work, network-flow telemetry is used as a proxy for Linux-host communication behavior, enabling the study of lateral movement patterns in a graph-based setting.

We present an unsupervised methodology based on a **Temporal Heterogeneous Graph Neural Network (THGNN)** to detect anomalous edges in a continuously evolving flow graph. The methodology converts raw network-flow records into a temporal heterogeneous graph in which edges are assigned one of five semantic types derived from flow characteristics and destination-port categories (normal, sensitive, scan-like, frequent, and crypto-like). A GRU-based temporal memory component with exponential decay, a Transformer-based walk encoder, a five-component unsupervised loss function, and a learnable rarity scorer jointly capture structural dependencies and temporal evolution. Due to dataset constraints in CIC-IDS2018 (which does not provide source IP addresses), source nodes are represented using behavioral flow signatures derived from traffic characteristics rather than individual host identities. The AIT-NDS dataset, in contrast, preserves endpoint identities and therefore allows direct host-level graph construction.

The proposed framework was evaluated on two publicly available network intrusion datasets, CIC-IDS2018 and AIT-NDS, representing different network environments and attack scenarios. Experimental results demonstrate competitive performance across ROC-AUC, Average Precision (AP), and F1-Score metrics while successfully surfacing complex attack propagations that are often missed by traditional monitoring systems. The consistent performance observed across both datasets highlights the robustness and generalizability of the proposed methodology.

Keywords: Temporal Heterogeneous Graph Neural Networks, Unsupervised Anomaly Detection, Lateral Movement Detection, Network Intrusion Detection, CIC-IDS2018, AIT-NDS, Edge Anomaly Scoring.

Résumé

Afin d'identifier les mouvements latéraux lors des cyberattaques, nous nous intéressons au problème de la détection des activités de propagation malveillante au sein des réseaux d'entreprise. Les attaquants exploitent fréquemment le mouvement latéral pour passer d'un hôte initialement compromis à d'autres ressources de grande valeur. La détection de ce comportement demeure difficile en raison du volume important du trafic réseau généré en continu ainsi que de l'évolution permanente des interactions entre les systèmes. Dans ce travail, la télémétrie des flux réseau est utilisée comme une approximation du comportement de communication des hôtes Linux, permettant ainsi l'étude des schémas de mouvement latéral dans un contexte fondé sur les graphes.

Nous proposons une méthodologie non supervisée basée sur un **Temporal Heterogeneous Graph Neural Network (THGNN)** afin de détecter les arêtes anormales dans un graphe de flux en évolution continue. La méthodologie transforme les enregistrements bruts des flux réseau en un graphe hétérogène temporel dans lequel les arêtes sont associées à l'un des cinq types sémantiques dérivés des caractéristiques des flux et des catégories de ports de destination (normal, sensible, de type balayage, fréquent et de type cryptographique). Un composant de mémoire temporelle basé sur les unités GRU avec décroissance exponentielle, un encodeur de parcours fondé sur les Transformers, une fonction de perte non supervisée à cinq composantes ainsi qu'un estimateur de rareté entraînable permettent conjointement de capturer les dépendances structurelles et l'évolution temporelle.

En raison des contraintes du jeu de données CIC-IDS2018, qui ne fournit pas les adresses IP sources, les nœuds sources sont représentés à l'aide de signatures comportementales dérivées des caractéristiques du trafic plutôt que par des identités d'hôtes individuelles. À l'inverse, le jeu de données AIT-NDS conserve les identités des terminaux, permettant ainsi une construction directe du graphe au niveau des hôtes.

Le cadre proposé a été évalué sur deux jeux de données publics de détection d'intrusions réseau, **CIC-IDS2018** et **AIT-NDS**, représentant différents environnements réseau et scénarios d'attaque. Les résultats expérimentaux démontrent des performances compétitives selon les métriques ROC-AUC, Average Precision (AP) et F1-Score, tout en mettant en évidence des propagations d'attaques complexes souvent ignorées par les systèmes de surveillance traditionnels. La cohérence des performances observée sur les deux jeux de données souligne la robustesse et la capacité de généralisation de la méthodologie proposée.

Mots-clés : Réseaux de Neurones sur Graphes Hétérogènes Temporels, Détection d'Anomalies Non Supervisée, Détection des Mouvements Latéraux, Détection d'Intrusions Réseau, CIC-IDS2018, AIT-NDS, Évaluation des Anomalies d'Arêtes.

الملخص

من أجل الكشف عن الحركة الجانبية أثناء الهجمات السيبرانية، نتناول مشكلة اكتشاف أنشطة الانتشار الخبيثة داخل شبكات المؤسسات. وغالبًا ما يستغل المهاجمون الحركة الجانبية للانتقال من مضيف تم اختراقه في البداية إلى موارد أخرى ذات قيمة عالية داخل الشبكة. ويُعد اكتشاف هذا السلوك تحديًا كبيرًا بسبب الحجم الهائل لحركة المرور الشبكية المتدفقة باستمرار، إضافةً إلى التطور المستمر للتفاعلات بين الأنظمة المختلفة. في هذا العمل، تُستخدم بيانات تدفقات الشبكة كمؤشر بديل لسلوك الاتصالات بين مضيفات نظام لينكس، مما يتيح دراسة أنماط الحركة الجانبية ضمن إطار قائم على الرسوم البيانية.

نقترح منهجية غير خاضعة للإشراف تعتمد على الشبكات العصبية البيانية الزمنية غير المتجانسة (THGNN) بهدف اكتشاف الحواف الشاذة في رسم بياني لتدفقات الشبكة يتطور باستمرار مع الزمن. نظرًا للقيود الموجودة في مجموعة البيانات، CIC-IDS2018 والتي لا توفر عناوين IP الخاصة بالمصادر، يتم تمثيل العقد المصدرية باستخدام بصمات سلوكية مشتقة من خصائص حركة المرور بدلاً من هويات المضيفات الفعلية. وعلى النقيض من ذلك، تحتفظ مجموعة البيانات AIT-NDS بهويات الأجهزة الطرفية، مما يسمح ببناء الرسم البياني مباشرةً على مستوى المضيفات.

تم تقييم الإطار المقترح باستخدام مجموعتي بيانات عامتين لاكتشاف اختراقات الشبكات، وهما CIC-IDS2018 وAIT-NDS، واللذان يمثلان بيانات شبكية وسيناريوهات هجومية مختلفة. أظهرت النتائج التجريبية أداءً تنافسيًا وفقًا لمقاييس ROC-AUC و Precision Average (AP) و F1-Score، مع القدرة على اكتشاف سلاسل انتشار هجومية معقدة غالبًا ما تقشل أنظمة المراقبة التقليدية في رصدها.

الكلمات المفتاحية: الشبكات العصبية البيانية الزمنية غير المتجانسة، اكتشاف الشذوذ غير الخاضع للإشراف، اكتشاف الحركة الجانبية، اكتشاف اختراقات الشبكات، CIC-IDS2018، AIT-NDS تقييم شذوذ الحواف.

Acronym	Definition
AIT-NDS	Advanced Information Technology Network Dataset
AP	Average Precision
APT	Advanced Persistent Threat
CPU	Central Processing Unit
CIC-IDS2018	Canadian Institute for Cybersecurity Intrusion Detection System 2018
GAT	Graph Attention Network
GCN	Graph Convolutional Network
GELU	Gaussian Error Linear Unit
GNN	Graph Neural Network
GRU	Gated Recurrent Unit
HEC	Heterogeneous Edge Convolution
HMM	Hidden Markov Model
HOLMES	Hunting Optimized Lateral Movement, Exposure, and Security
IDS	Intrusion Detection System
InfoNCE	Information Noise Contrastive Estimation
IP	Internet Protocol
LSTM	Long Short-Term Memory
MITRE	MITRE ATT&CK Framework
MLP	Multi-Layer Perceptron
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
RNN	Recurrent Neural Network
SIEM	Security Information and Event Management
SSH	Secure Shell
TCP	Transmission Control Protocol
TGN	Temporal Graph Network
THGNN	Temporal Heterogeneous Graph Neural Network
UDP	User Datagram Protocol

Contents

Introduction	12
1 Domain Definition and Background	14
1.1 Introduction	14
1.2 Linux Security and the Threat Landscape	14
1.2.1 Linux as the Dominant Enterprise OS	14
1.2.2 Cyber Security Taxonomy	14
1.3 Lateral Movement — Definition and Stages	14
1.3.1 Linux-Relevant MITRE Techniques	15
1.3.2 Real-World Incidents	16
1.4 Linux Audit and Logging Infrastructure	16
1.4.1 Audit Subsystem — auditd, syslog, journald	16
1.4.2 System Call Telemetry	17
1.4.3 EDR Tools — Falco, osquery, Sysdig	17
1.4.4 Provenance Graphs	17
1.4.5 Log Volume Challenges	17
1.5 Deep Learning Foundations	18
1.5.1 Machine Learning Paradigms	18
1.5.2 Neural Networks	18
1.5.3 Convolutional Neural Networks (CNNs)	18
1.5.4 Recurrent Networks (RNN / LSTM)	18
1.5.5 Attention Mechanism and Transformers	18
1.6 Conclusion	19
2 Graph Neural Networks for Security Analysis	20
2.1 Introduction	20
2.2 Graph Theory Fundamentals	20
2.2.1 Basic Definitions	20
2.2.2 Homogeneous vs. Heterogeneous Graphs	20
2.2.3 Temporal and Dynamic Graphs	20
2.2.4 Provenance Graph Construction from Audit Logs	21
2.2.5 Graph Anomaly Detection Granularity	21
2.3 Message-Passing Framework	21
2.4 Graph Convolutional Networks (GCN)	22
2.4.1 Principle	22
2.4.2 Application to Security	22
2.4.3 Advantages	22

2.4.4	Limitations	22
2.5	Graph Attention Networks (GAT)	23
2.5.1	Principle	23
2.5.2	Application to Security	23
2.5.3	Advantages	23
2.5.4	Limitations	23
2.6	GraphSAGE	24
2.6.1	Principle	24
2.6.2	Application to Security	24
2.6.3	Advantages	24
2.6.4	Limitations	24
2.7	Temporal Graph Networks (TGN)	25
2.7.1	Principle	25
2.7.2	Application to Security	25
2.7.3	Advantages	25
2.7.4	Limitations	25
2.8	DyRep	26
2.8.1	Principle	26
2.8.2	Application to Security	26
2.8.3	Advantages	26
2.8.4	Limitations	26
2.9	Temporal Heterogeneous Graph Neural Networks(THGNN)	27
2.9.1	Principle	27
2.9.2	Application to Security	27
2.9.3	Advantages	27
2.9.4	Limitations	27
2.10	Summary Comparison of GNN Architectures	28
2.11	Conclusion	28
3	Problem Definition and State of the Art	30
3.1	Introduction	30
3.2	Formal Problem Definition	30
3.2.1	System Model	30
3.2.2	Provenance Graph	30
3.2.3	Detection Objective	31
3.2.4	Constraints and Challenges	31
3.3	State of the Art	32
3.3.1	Rule-Based and Signature-Based Systems	32
3.3.2	Statistical and Machine-Learning Anomaly Detection	32

3.3.3	Deep Sequence Models — LSTM-Based Detection	33
3.3.4	HOLMES — Provenance Graph Correlation for APT Detection . .	34
3.3.5	MCI — Causality Inference for Accurate Provenance Construction	35
3.3.6	ATLAS — Sequence-Based Attack Investigation	35
3.3.7	THREATTRACE — Node-Level Provenance Graph Learning	36
3.4	Synthesis and Identified Gaps	37
3.5	Conclusion	37
4	Proposed Approach	39
4.1	Introduction	39
4.2	Network Flow Data as a Proxy for Audit Telemetry	39
4.3	Graph Construction — CIC-IDS2018	40
4.3.1	Dataset	40
4.3.2	Node Types and Graph Structure (CIC-IDS2018)	40
4.4	Edge Feature Vector (CIC-IDS2018)	40
4.5	Graph Construction — AIT-NDS	42
4.5.1	Dataset	42
4.5.2	Label Vocabulary	42
4.5.3	Node Identities and Graph Structure (AIT-NDS)	43
4.5.4	Edge Feature Vector (AIT-NDS)	44
4.6	Dataset Comparison	47
4.7	Model Architecture — THGNN	48
4.7.1	Node Projection and Memory Fusion	49
4.7.2	Temporal Node Memory	49
4.7.3	Heterogeneous Edge Convolution (HEC)	50
4.7.4	Memory-Aware Walk Generator	50
4.7.5	Transformer Walk Encoder	51
4.7.6	Edge Scorer	51
4.8	Training Strategy	51
4.8.1	Unsupervised Objective	51
4.8.2	Optimization	52
4.8.3	Streaming Training Procedure	53
4.9	Hyperparameter Tuning	54
4.9.1	Search Space	54
4.9.2	Tuning Strategies	55
4.9.3	Temporal Window Analysis	56
4.9.4	Selection Criterion and Final Configuration	56
4.10	Inference and Anomaly Scoring	57
4.10.1	Edge-Level Anomaly Score	57

4.10.2	Path-Level Lateral Movement Scoring	57
4.10.3	Threshold Selection	58
4.11	Experimental Validation Design	58
4.11.1	Evaluation Protocol	58
4.11.2	Ablation Study Design	58
4.12	Summary of Design Choices	58
4.13	Conclusion	61
5	Implementation and Results	62
5.1	Introduction	62
5.2	Experimental Setup	62
5.2.1	Hardware and Software Environment	62
5.2.2	Dataset	62
5.2.3	Model Configuration	63
5.2.4	Baselines	63
5.2.5	Evaluation Metrics	64
5.3	Results and Analysis	64
5.3.1	Main Detection Performance	64
5.3.2	Training Dynamics	65
6	Training Convergence	66
6.0.1	Ablation Study	66
6.0.2	Path-Level Lateral Movement Detection	67
6.1	Discussion	68
6.1.1	Strengths of the Proposed Framework	68
6.1.2	Limitations and Failure Cases	69
6.1.3	Comparison with Prior Art	69
6.2	Conclusion	70
	Conclusion	72

Introduction

Linux has been expanding rapidly throughout enterprise infrastructures — from cloud-native servers and containerized applications to large-scale HPC clusters — becoming both the backbone of modern IT and a valuable target for sophisticated attackers. One of the most dangerous phases of a cyber-attack is *lateral movement*: a systematic method by which adversaries expand control across a network, pivoting from one host to another until a desired target is reached. Unlike the initial breach, lateral movement is specifically designed to blend into normal system behavior by reusing legitimate user credentials, standard system utilities, and trusted internal services — a strategy commonly referred to as Living Off The Land (LOTL) (7).

Detection of lateral movement in Linux is especially difficult for several reasons. First, the Linux Audit Subsystem generates tremendous volumes of telemetry (system calls, process spawns, file accesses) at rates that render manual analysis impossible. Second, individual events within an attack chain may appear benign in isolation; their malicious nature becomes apparent only when considered collectively — as a sequence of entity interactions and timings.

Current solutions include rule-based and signature-based systems such as Falco and Snort. These are fast and interpretable but inherently reactive: they detect only previously cataloged attack patterns and are ineffective against novel techniques, LOTL campaigns, and zero-day exploits. Statistical and machine-learning methods provide better generalization but require significant manual feature engineering and produce unacceptable false-positive rates in operational environments. Sequence models based on Recurrent Networks reason about individual process sequences but cannot model the multi-entity, multi-hop relational structure characteristic of lateral movement. Even advanced provenance-graph approaches such as HOLMES (8) and THREATTRACE (27) rely on manually crafted attack templates or static graph snapshots.

The central premise of this research is that effective lateral movement detection requires a model that jointly captures:

1. **Structural heterogeneity** — distinct node types and semantically different edge relations;
2. **Temporal dynamics** — exact ordering and timing of events; and
3. **Behavioral context** — each node’s historical behavior relative to its baseline.

Based on the literature reviewed in this thesis, no existing approach was found to simultaneously satisfy all three criteria.

Scope and dataset. The implementation presented in this thesis operates on network flow records drawn from the CIC-IDS2018 dataset (32). While kernel-level audit logs are the

ideal data source for Linux lateral movement detection, CIC-IDS2018 serves as a publicly available, labeled benchmark that enables reproducible evaluation and comparison with prior art. The relationship between network flow data and Linux audit telemetry is discussed in Section 4.2. Because CIC-IDS2018 does not provide source IP addresses, source nodes are represented as behavioral fingerprints rather than individual hosts; this limitation affects the granularity of host-level lateral movement detection and is explicitly acknowledged throughout.

The proposed framework is trained in a fully unsupervised manner, requiring no ground-truth attack labels during training, which mirrors real-world deployment conditions.

Thesis organization. Chapter 1 provides domain context: the Linux security landscape, stages of lateral movement with MITRE ATT&CK mappings, the Linux audit infrastructure, and foundational deep-learning concepts. Chapter 2 reviews GNN architectures applicable to security analysis. Chapter 3 formally defines the problem and surveys eight categories of prior work. Chapter 4 specifies the complete framework. Chapter 5 reports implementation details and empirical results.

1 Domain Definition and Background

1.1 Introduction

The purpose of this chapter is to provide the background necessary to address lateral movement detection in Linux-based security environments. This includes the current Linux threat landscape, taxonomy of relevant attacks, the Linux audit infrastructure, and the deep-learning concepts underpinning the proposed framework. A separate treatment of graph theory and GNN architectures is provided in Chapter 2.

1.2 Linux Security and the Threat Landscape

1.2.1 Linux as the Dominant Enterprise OS

During the last decade Linux became the primary server operating system used by large enterprises, hosting websites, cloud services, and containerized applications at scale (1). Its open-source model, long-term supportability, and ubiquity in data centers make it both the backbone of modern IT infrastructure and a prime target for sophisticated adversaries (4).

1.2.2 Cyber Security Taxonomy

Three attack categories are most relevant to this research.

Advanced Persistent Threats (APTs) Sophisticated, often state-sponsored intrusion campaigns designed to gain unauthorized access to high-value targets over a prolonged period (5). Attackers continuously adapt their methods to mimic legitimate network activity.

Insider threats Authorized personnel who access or misuse sensitive information, either intentionally or through negligence. Traditional defense-in-depth models often fail to detect such threats because the actors hold valid credentials.

Supply-chain attacks Adversaries compromise a dependent component, build process, or supplier rather than targeting an organization directly. According to the Verizon 2024 Data Breach Investigations Report (DBIR), third-party involvement accounts for approximately 15 % of all breaches, an increase of 68 % over the previous year (2).

1.3 Lateral Movement — Definition and Stages

When an attacker gains a foothold in a network, lateral movement represents the methods used to extend access from an initially compromised device to other systems and high-value assets (4). According to the MITRE ATT&CK framework, lateral movement is documented

as tactic TA0008, encompassing Remote Service Exploitation, Credential Reuse, and Internal Spear-phishing. A complete intrusion campaign typically proceeds through the following stages (4):

1. Initial Access (phishing or exploitation of public-facing applications);
2. Establishment of execution and persistence;
3. Privilege Escalation / Credential Access;
4. Discovery and lateral movement to additional systems via remote services;
5. Collection, Staging, and Exfiltration.

Milajerdi et al. illustrated that correlating suspicious information flows across all stages via provenance graphs enables real-time APT detection that event-based rule sets consistently miss (8).

1.3.1 Linux-Relevant MITRE Techniques

The MITRE ATT&CK framework defines several techniques that adversaries commonly employ for lateral movement in Linux environments, as summarized in Table 1.3.1.

ID	Name	Description
T1021	Remote Services	Adversaries use valid accounts to access remote systems via services commonly used in Linux environments.
T1021.004	SSH	Remote login via SSH using valid credentials.
T1021.005	VNC	Remote desktop access via VNC in Linux environments.
T1021.007	Cloud Services	Lateral movement through cloud-based services.
T1021.008	Direct Cloud VM Connections	Direct access to cloud-hosted Linux virtual machines.
T1570	Lateral Tool Transfer	Transferring tools via SCP or rsync between Linux systems.

Continued on next page

Table 1 continued from previous page

ID	Name	Description
T1210	Exploitation of Remote Services	Exploiting vulnerabilities in services like SSH or web servers.
T1550	Use Alternate Authentication Material	Using stolen SSH keys or tokens.
T1550.004	Web Session Cookie	Using stolen session cookies for access.

1.3.2 Real-World Incidents

- **SolarWinds (2020):** Attackers compromised the SolarWinds Orion build pipeline to distribute a backdoor update, then moved laterally through on-premises and cloud environments for months (3). The supply-chain entry point and long dwell time make this a canonical APT example.
- **NotPetya (2017):** Rapid lateral propagation exploiting EternalBlue and Mimikatz credential harvesting caused an estimated \$10 billion in global damage (6).
- **Volt Typhoon (2023):** A state-sponsored campaign against critical infrastructure relying exclusively on LOTL techniques — legitimate system tools and valid credentials — rather than custom malware (7).

Each of the aforementioned attacks utilize the same technique to avoid detection by the defender. Each utilized legitimate system activity so as to blend in. Using legitimate system activity allows the defender to focus on detecting patterns of behavior as opposed to focusing on specific alert(s).

1.4 Linux Audit and Logging Infrastructure

1.4.1 Audit Subsystem — auditd, syslog, journald

Linux provides multiple mechanisms for recording system activity (9):

- **auditd:** The Linux Audit Daemon records kernel-level system calls (e.g., `execve`, `open`) with high granularity, making it the primary data source for host-based forensics.
- **syslog / journald:** Handle user-space application logs and kernel messages at coarser granularity; useful for correlating cross-service interactions during incident response.

1.4.2 System Call Telemetry

The most security-relevant system calls include (10):

- `execve` — reveals each new process spawned by another, enabling tracking of malicious execution chains.
- `connect` / `accept` — reveals network communication (who talks to whom).
- `open`, `read`, `write` — reveals file access patterns (e.g., access to `/etc/shadow`).
- `clone` / `fork` — child-process spawning, which may indicate process-injection attempts.

1.4.3 EDR Tools — Falco, osquery, Sysdig

- **Falco** (11) applies rule-based filtering to the syscall stream to flag behavioral anomalies in real time.
- **osquery** exposes the OS as a relational database, allowing SQL queries over running processes, network connections, and loaded modules.
- **Sysdig** (12) provides deep system-call capture, particularly valuable for container-level forensics.

The common limitations of these tools are they are generally reactive — identifying and triggering alerts based on individually identified bad behaviors and/or signature matching versus identifying behavioral anomalies through a sequence of events.

1.4.4 Provenance Graphs

A *provenance graph* is a directed graph in which each node represents a system entity (process, file, socket) and each edge represents an information flow between entities (e.g., a process reading a file or establishing a network connection) (8). Modeling audit trails as provenance graphs and correlating flows within those graphs allows identification of APT phases that flat log analysis consistently misses (8). The causal structure of provenance graphs makes it significantly harder for an attacker to erase evidence without creating structural anomalies, forming the conceptual foundation for the framework proposed in Chapter 4.

1.4.5 Log Volume Challenges

Enterprise generated audit logs create massive amounts of data. Alsaheel et al. Noted that one of the largest obstacles to developing effective attack detection systems utilizing system-wide telemetry is the sheer Volume and noise of system-generated logging data, in that most

attack signals are hidden within vast amounts of benign events (13). Therefore, the results are logical: alert fatigue, failure to correlate causally related events, and the ability for an attacker to blend into a typical-looking syscall sequence.

1.5 Deep Learning Foundations

1.5.1 Machine Learning Paradigms

In supervised machine learning models are trained on labeled examples of both benign and malicious activity, however the creation of labeled training sets is often very costly. In unsupervised machine learning models normal activity is modeled and flagged as unusual — appealing for security applications due to the relative lack of labeled training data, however tend to generate higher rates of false alarms (19; 13).

1.5.2 Neural Networks

Standard feed-forward neural networks map inputs to outputs by applying multiple layers of linear transformations, followed by non-linear activation functions such as ReLU or sigmoid. Prediction errors are used to adjust weights during back-propagation (19).

1.5.3 Convolutional Neural Networks (CNNs)

CNNs utilize the same learned filters across all spatial locations within the input image (20). Once “neighborhood” is formally defined the CNN paradigm also applies to graphs giving rise to graph convolutional networks — described in detail in chapter 2.

1.5.4 Recurrent Networks (RNN / LSTM)

RNNs & LSTMs provide reasonable processing of time-series logs and sequential data (19), however their inherent limitation is that they treat sequences of data as opposed to treating them as graphs and therefore do not account for the spatial relationships between system entities that are fundamental to detecting attacks based on provenance.

1.5.5 Attention Mechanism and Transformers

The Attention mechanism enables a model to dynamically assign weighting factors to different portions of the input (21). Although the full transformer architecture can become computationally prohibitive for large graphs the Attention mechanism itself can be applied to graph-based architectures explored in chapter 2.

1.6 Conclusion

This chapter provided a comprehensive introduction to required background knowledge for the remainder of this dissertation. This included describing the threat landscape facing Linux, explaining lateral movement — including its various stages, reviewing auditing features present on Linux platforms and providing descriptions of key deep-learning concepts that the proposed approach will rely upon. Graph-theoretic fundamentals and GNN architectures will be described in detail in chapter 2, along with a formal description of the problem being addressed and a survey of prior art solutions in Chapter 3.

2 Graph Neural Networks for Security Analysis

2.1 Introduction

The purpose of this chapter is to provide a foundation in both the mathematics and architecture of Graph Neural Networks (GNNs) as applied to security analysis of audit log provenance data. We will first introduce the basic graph theory used by GNNs, then provide detailed descriptions of each major architecture of GNNs, and finally explain how provenance graphs are constructed from raw audit log events to be used as model inputs. Each architecture will be described in terms of increased complexity from simple, static homogeneous models through to complex, dynamic heterogeneous temporal networks.

2.2 Graph Theory Fundamentals

2.2.1 Basic Definitions

A graph $G = (\mathcal{V}, \mathcal{E})$ consists of a vertex set $\mathcal{V}$ and an edge set $\mathcal{E}$ of directed pairs over $\mathcal{V}$. The adjacency matrix $A \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ encodes connectivity; combined with the degree matrix D , it yields the *unnormalized* graph Laplacian $L = D - A$. For spectral GCN methods the symmetric normalized Laplacian $\tilde{L}_{\text{sym}} = D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$ is used instead of the unnormalized form (15; 22).

2.2.2 Homogeneous vs. Heterogeneous Graphs

All nodes in homogeneous graphs are considered to be equivalent. However, as provenance graphs include processes, files and sockets, each represented by distinct types of nodes that are interconnected by multiple types of edges (8; 17); therefore, they represent **heterogeneous graphs**. Furthermore, this heterogeneity also carries critical information that a detection system should take advantage of. For example, an edge denoting an `execve` call conveys significantly different security semantics than an edge denoting a read access to a credential file.

However, many GNN architectures treat all edge types as equivalent (as illustrated in the comparison table in Section 2.10). Therefore, this omission leads to the development of the Heterogeneous Temporal GNN architecture discussed in Section 2.9 that explicitly includes both node types and edge semantics in its message-passing procedure.

2.2.3 Temporal and Dynamic Graphs

Audit logs are continuously generated over time. A temporal graph embeds timestamps into edges (18), so in addition to noting whether a relationship exists between two objects, it also

records when the relationship existed. For example, if there was an SSH connection established at 2AM from a user account whose normal operational hours were between 9-5PM, it would convey a significantly greater risk than an otherwise identical connection initiated at noon. As such, temporal structure represents the mechanism for recording differences in risk profiles.

2.2.4 Provenance Graph Construction from Audit Logs

For each audit event, a directed edge is created between a subject (in nearly all cases, a process) and an object (a file, socket or other process), with an attribute label assigned to the edge indicating the type of system call associated with the event along with a timestamp (14). Chaining these audit events creates a causal graph illustrating who executed what program(s), which files were accessed and which network connections were established and in what order. Kwon et al., demonstrated that inferring precise causality relationships between individual system calls rather than coarse dependencies between processes is necessary to create accurate and concise attack graphs (14).

2.2.5 Graph Anomaly Detection Granularity

Graph anomalies can occur at three different levels of detail (16):

- **Node-level:** Is this specific process exhibiting unusual behavior relative to similar processes?
- **Edge-level:** Is this system call between these two subjects abnormal given their historical interactions?
- **Subgraph-level:** Does this group of behaviors taken collectively resemble a previously identified attack pattern?

The majority of lateral movement detection occurs at the sub-graph level. While individual actions within an attack chain may appear benign individually, it is only once the entire sequence of attacks becomes evident does the overall attack become identifiable.

2.3 Message-Passing Framework

Every GNN variant builds on the same base: in each layer, each node’s representation is updated by aggregating messages from its neighbors (25). After k rounds of message-passing the embedding of a node contains information about its neighborhood within a radius of k hops:

$$h_v^{(k)} = \text{UPDATE}^{(k)}\left(h_v^{(k-1)}, \text{AGG}^{(k)}\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}\right). \quad (1)$$

The architectures reviewed below (GCN, GAT, GraphSAGE, TGN, DyRep, and THGNN) differ in how they specify AGG and UPDATE, and whether those specifications account for edge types, edge timestamps, or both.

2.4 Graph Convolutional Networks (GCN)

2.4.1 Principle

Kipf & Welling (22) proposed the use of a spectral convolution based on graph signal processing in GCN. For each layer, the algorithm computes:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} H^{(l)} W^{(l)}\right)$$

where $\tilde{A} = A + I_N$ adds self-loops and $\tilde{D}$ normalizes by node degree. The propagation of node features is done symmetrically over neighbors using a single shared weight matrix $W^{(l)}$ per layer.

2.4.2 Application to Security

In a provenance graph, GCN spreads process, file, and socket features throughout the graph enabling a process node to collect contextual information concerning its communication partners and accessed files. Global contextualization can be helpful in identifying if a process belongs to a known attack sequence.

2.4.3 Advantages

- Very simple and computationally inexpensive.
- Is effective in capturing local neighborhood structure.
- There are good theoretical foundations related to spectral graph theory.

2.4.4 Limitations

- It treats all neighbors equally so cannot differentiate, e.g., between critical SSH socket edges versus routine file read edges.
- Transductive: requires the entire graph to be available during training time. Not usable with stream-based audit data that has continuous creation of new processes.
- Has limited expressiveness for modeling edge-types and/or edge-timestamps on heterogeneous temporal graphs.

2.5 Graph Attention Networks (GAT)

2.5.1 Principle

GAT (23) introduces learnable per-neighbor attention coefficients:

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \| W\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [W\mathbf{h}_i \| W\mathbf{h}_k]))}. \quad (2)$$

The aggregated representation is $\mathbf{h}'_i = \sigma(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} W\mathbf{h}_j)$; multi-head attention can be applied for training stability.

2.5.2 Application to Security

On provenance graphs the attention mechanism enables the model to pay greater attention to suspicious edges, such as paying much more attention to a seldomly occurring outbound connection than paying great attention to many thousand routine file reads. This is especially helpful when the few attack-related edges represent only an infinitesimal fraction of the total number of benign activities.

2.5.3 Advantages

- The attention mechanism adaptively weights neighbors increasing sensitivity to sparse-but-security-relevant edge-types.
- Much more interpretable than GCN: the attention-weights directly indicate what relationships the model finds most relevant.
- Can be applied inductively since the attention-mechanism is shared across all nodes.

2.5.4 Limitations

- Has higher computational costs than GCN as there is an attention-coefficient computation required for each edge.
- Temporal aspects of the edges in a particular snapshot are still ignored; all edges in a particular snapshot are treated as simultaneous.
- On extremely dense graphs performance may degrade due to diffuse attention-weights.

2.6 GraphSAGE

2.6.1 Principle

GraphSAGE (24) improves scalability and inductiveness by sampling a fixed-size neighborhood and learning an aggregation function:

$$\mathbf{h}_v^{(k)} = \sigma\left(W^{(k)} \cdot \text{CONCAT}\left(\mathbf{h}_v^{(k-1)}, \text{AGG}\left\{\mathbf{h}_u^{(k-1)} : u \in \mathcal{S}(v)\right\}\right)\right), \quad (3)$$

where $\mathcal{S}(v)$ is a random sample of $\mathcal{N}(v)$. The aggregator may take the form of average, max-pooling functions, or LSTMs.

2.6.2 Application to Security

GraphSAGE is ideal for real-time security monitoring platforms such as intrusion detection systems. Because GraphSAGE learns a generalized aggregation function versus specific node embeddings, GraphSAGE can generate vector representations of nodes that were not observed during training without requiring model re-training.

2.6.3 Advantages

- *Inductive*: represents unknown nodes; necessary for real-time detection on live systems.
- *Scalability*: handles extremely large graphs using neighbor sampling techniques.
- *Flexibility*: ability to choose multiple forms of aggregation based on characteristics of the graph.

2.6.4 Limitations

- Neighbor sampling produces randomness in the generated embeddings, potentially decreasing consistency among inference runs.
- Does not capture the temporal sequence of edges..
- Diverse edge types require significant programming (e.g. use of a different aggregator for each unique edge-type).

The three architectures described above (GCN, GAT and GraphSAGE) are designed to operate on static snapshots of graphs. These architectures consider all edges simultaneously when processing the input graph while ignoring the sequential order of audit events occurring within the graph. Two subsequent architectures extend these ideas by modeling the graph as a continuous time dynamic system.

2.7 Temporal Graph Networks (TGN)

2.7.1 Principle

Temporal Graph Networks (28) build upon the message passing paradigm to include continuous time dynamic graphs. At each point in time t , TGNs memory component $s_v(t)$ is updated with the arrival of a new event involving the node:

$$\mathbf{s}_v(t) = \text{mem}(\mathbf{m}_v(t), \mathbf{s}_v(t^-)), \quad (4)$$

where $m_v(t)$ is a message calculated from the interacting nodes and the elapsed time since the previous update. A graph attention mechanism then generates temporal embeddings from the memory state vectors.

2.7.2 Application to Security

TGN tracks *behavioral changes*: recall the 2AM SSH connection from Section 2.2 – TGN uses its memory components to keep track of how many times a particular node has previously sent an SSH request. Therefore, if a node receives an unusual number of requests, the system identifies this as deviating from the node’s expected behavior relative to global norms. As such, this concept is highly applicable to living off the land attacks. Malicious activities will blend into the normal operations of the system tools and cannot be distinguished unless there is some deviation in their operation compared to what would normally occur.

2.7.3 Advantages

- Event processing occurs in continuous time maintaining the causal ordering of audit events.
- Node memories allow for the identification of slight changes in long term behaviors.
- Maintains benefits of temporal modeling along with graph structured representation learning.

2.7.4 Limitations

- Processing memory updates sequentially per node limits the throughput under high event rates.
- More complicated to train and fine-tune than static GNNs.
- Nodes’ memory components can become outdated and/or contain inaccurate/ noisily acquired information after extended periods of time.

2.8 DyRep

2.8.1 Principle

DyRep (26) represents changes in a node’s representation through a **temporal point process**. In other words, DyRep simulates two independent processes: (i) how do nodes communicate? (ii) how does the graph evolve structurally? Updates to a node’s representation are expressed as:

$$\mathbf{z}_v(t) = \Phi(\mathbf{z}_v(t^-), \text{context}_v(t), \Delta t), \quad (5)$$

where Δt is the amount of time that has passed since the last event occurred and Φ is a gated recurrent unit.

2.8.2 Application to Security

The dual-process nature of DyRep provides value in representing two fundamentally different things in provenance graphs: (i) frequently short-lived communications (individual syscalls); (ii) less-frequent structural changes (new persistent links between previously unrelated hosts), which are indicative of lateral movement. While DyRep has too much computation involved to serve as my primary detection architecture, I include DyRep as a principled temporal baseline as it explicitly separates the evolving dynamics of node-to-node communications from the evolving structure of the network, providing an interesting theoretical comparison to TGN’s memory-based approach, and to THGNN’s unified heterogeneous formulation, discussed below.

2.8.3 Advantages

- Represents both communication frequencies and structural evolution simultaneously.
- Encapsulates temporal delta values enabling the model to respond to irregularities in timing between events.
- Provides a probabilistic framework supporting uncertainty estimations.

2.8.4 Limitations

- Has higher computational complexity than TGN due to its point-process formulation.
- Requires tuning of both sub-processes that can be challenging given limited labeled examples.
- Not as well supported by existing graph-learning libraries as GCN/GAT, increasing implementation overhead.

2.9 Temporal Heterogeneous Graph Neural Networks(THGNN)

2.9.1 Principle

THGNN addresses both heterogeneity and temporal dynamics simultaneously through type-aware message passing and continuous-time event processing. The key idea is to assign a separate transformation for each $(src_type, edge_type, dst_type)$ triplet. The UPDATE rule for a node v of type $\phi(v)$ at time t is:

$$\mathbf{h}_v^{(k)}(t) = \text{update}_{\phi(v)} \left(\mathbf{h}_v^{(k-1)}(t), \bigoplus_{\tau \in \mathcal{T}_E} \text{agg}_{\tau} \left(f \left(\mathbf{h}_u^{(k-1)}(t), \Delta t_{uv}, \mathbf{r}_{uv} \right) : (u, v, \tau) \in \mathcal{E}(t) \right) \right), \quad (6)$$

where $\mathcal{E}(t)$ is the edge set at time t , $\bigoplus_{\tau}$ denotes type-specific aggregation, Δt_{uv} is the time elapsed since the last interaction on edge (u, v) , and $\mathbf{r}_{uv}$ is the relation-type embedding.

2.9.2 Application to Security

THGNN directly addresses the limitations of all prior architectures. Type-specific aggregators allow the model to learn that an `execve` edge to a shell binary is fundamentally different from a `connect` edge to an external IP address. Temporal encoding via Δt_{uv} provides sensitivity to anomalous timing: a dormant process that suddenly establishes a network connection after a long period of inactivity generates a large time-delta signal that static architectures cannot detect. Adversaries using LOTL techniques (e.g., Volt Typhoon) create no malware signatures but generate an unusual combination of heterogeneous edges at an unusual moment — precisely the signal is designed to amplify.

2.9.3 Advantages

- jointly models node Type, edge Type and temporal dynamics — the three structural properties most important when analyzing provenance graphs.
- Type-specific aggregators avoid collapsing semantically different system call types into one shared weight matrix.
- captures behavioral-drift via temporal-recency-encoding without having a dedicated memory-module.
- inductive by design: Type-conditioned-aggregators generalize to unseen nodes of known types thereby allowing for real time deployment.

2.9.4 Limitations

- number of learnable parameters increases with number of $(node_type, edge_type)$ -combinations, thus substantially increasing memory usage when dealing with large

vocabularies of node and edge types.

- data preparation: requires time-stamped training-data sets with both temporal order as well as labels for node and edge types. More demanding on data-preparation than homogeneous models.
- interpretability: more complex than GAT: while attention weights are available again, the interaction between Type-specific aggregators and temporal-encoding makes attribution less clear.

The formulation above describes a generic heterogeneous temporal graph framework. In this thesis, THGNN is adapted to network-flow graphs, where nodes correspond to hosts and edges correspond to communication flows.

2.10 Summary Comparison of GNN Architectures

Taken together, the six architectures account for the primary architectural decisions involved in detecting lateral movements based on provenance: static vs. Temporal, transductive vs. Inductive and uniform vs. Type-aware aggregation. Table 2 presents a summary of the respective trade-offs.

Architecture	Temporal	Inductive	Heterogen	Scalability	Complexity
GCN	No	No	Limited	High	Low
GAT	No	Partial	Limited	Medium	Medium
GraphSAGE	No	Yes	Limited	High	Low
TGN	Yes	Yes	Partial	Medium	High
DyRep	Yes	Yes	Partial	Low	High
THGNN	Yes	Yes	Full	Medium	High

Table 2: Comparison of GNN architectures along dimensions relevant to provenance-based detection.

2.11 Conclusion

The foundational elements behind the operation of gnns have been reviewed using a graph theory lens. Six key architectures (GCNs, GAT, GraphSAGE, TGN, DyRep, and THGNN) were detailed. They all make differing trade-offs concerning their ability to provide both expressive power and scalable solutions and to recognize temporal and heterogeneous relationships. As demonstrated in Table, THGNN is best suited for detecting lateral movement across provenance graphs due to its ability to combine type-aware message passing with continuous-time event processing. The formal definition of the lateral movement detection

problem and a survey of previous applications of similar methodologies to intrusion detection will follow in the subsequent chapters.

3 Problem Definition and State of the Art

3.1 Introduction

In addition to providing a formal mathematical formulation of the lateral movement detection problem based upon the prior chapters’ intuitive descriptions of the problem, this chapter presents a thorough review of previously proposed solution methodologies. These include traditional rule-based methods and the latest graph learning approaches to intrusion detection. An individual assessment of each methodology along with an evaluation of the strengths and limitations of each methodology will allow us to clearly articulate the motivations for our contributions in Chapter 4.

Although Chapter 2 presents provenance-graph-based graph neural networks as the theoretical foundation for lateral movement detection, the proposed THGNN framework is instantiated on network-flow graphs derived from the CIC-IDS2018 and AIT-NDS datasets. Consequently, nodes represent network hosts and edges represent communication flows rather than process–file interactions. This adaptation preserves the core principles of heterogeneous temporal graph learning while enabling evaluation on widely used network intrusion detection benchmarks.

3.2 Formal Problem Definition

3.2.1 System Model

Let $\mathcal{E} = \{e_1, e_2, \dots, e_n\}$ denote a stream of audit events in order of increasing timestamp. For each event $e_i = (s_i, o_i, r_i, t_i)$, s_i denotes a subject (i.e., process) performing a system call of type r_i on an object (file/socket/process) o_i at time t_i .

3.2.2 Provenance Graph

We construct a heterogeneous temporal provenance graph $G = (\mathcal{V}, \mathcal{E}, \mathcal{T}_V, \mathcal{T}_E, \phi, \psi)$ from this stream, where:

- $\mathcal{V}$ — nodes representing system entities (processes, files, sockets);
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ — directed edges derived from audit events;
- $\mathcal{T}_V, \mathcal{T}_E$ — sets of node and edge types, respectively;
- $\phi : \mathcal{V} \rightarrow \mathcal{T}_V$ — node-type mapping;
- $\psi : \mathcal{E} \rightarrow \mathcal{T}_E \times R_+$ — edge-type and timestamp mapping.

Each node $v \in \mathcal{V}$ carries a feature vector $\mathbf{x}_v \in R^d$ encoding measurable attributes.

3.2.3 Detection Objective

Let $G = \{G^{(1)}, G^{(2)}, \dots\}$

denote a sequence of heterogeneous temporal graph snapshots generated from audit events.

The objective is to learn a function

$$f_\theta : G \rightarrow R^{|E|}$$

that assigns an anomaly score to each interaction (edge) in the graph.

For an edge $e=(u,v,t)$,

$$s_e = f_\theta(e)$$

where larger values indicate a higher likelihood that the interaction is associated with malicious activity or lateral movement behavior.

Node representations are learned through temporal heterogeneous message passing and are subsequently used to score interactions and attack paths.

3.2.4 Constraints and Challenges

There exist several constraints governing the learning problem:

- **Class imbalance:** attack events constitute less than one percent of the total number of audit events in any operational setting.
- **Concept drift:** behavior exhibited by attackers changes overtime; i.e., the distribution of malicious subgraphs at deployment time may differ significantly from the training data.
- **Heterogeneous data:** node and edge semantics are modeled jointly because they belong to different categories.
- **Temporal relationships:** the temporal ordering of edges carries causal information that needs to be preserved during modeling.
- **Scale:** graphs may comprise tens-of-millions of nodes and edges; thus limiting the selection of models and inference algorithms.

Therefore, the central research question becomes: can a function f_θ be defined that exploits the structural/heterogeneous/temporal characteristics of provenance graphs so that it achieves acceptable levels of detection accuracy and false positive rates at large-scale enterprise environments?

3.3 State of the Art

3.3.1 Rule-Based and Signature-Based Systems

Description: Most popular IDS software like Snort and Suricata are based on matching a packet or log entry against a predefined signature list. Similarly, Rule-based correlation engines (e.g. In SIEM systems) will raise alarms if there exists a combination of events which correspond to a logical condition written into the engine by a user.

Advantages:

- the fastest and lightest systems possible (can run on commodity hardware) to analyze Packets/audit entries in real-time.
- each alarm raised corresponds to a specific, identifiable Rule. This makes forensics Much simpler than correlating alarms generated by complex statistical methods.
- no false alarms when analyzing the exact same variant of an attack for which a signature has been defined.

Limitations:

- blind to new types of attacks, referred to as "zero-day" attacks.
- manual effort required to update signature databases, this requires significant money and takes much longer to implement changes compared to how quickly attackers Can create new exploits.
- only analyzes each audit entry independently and/or as a sequence of events, Therefore it cannot reason about lateral movement, which involves moving through multi-hops of relational connections between hosts.
- as lateral movement exploits utilize legitimate tools and valid user credentials To move through the network they produce no signature matches.

3.3.2 Statistical and Machine-Learning Anomaly Detection

Description: A lot of research attempted to replace handcrafted signatures with statistical models of what constitutes normal behavior and flag deviations from that normal behavior as anomalies. Some early works used Hidden Markov Models (HMMs) trained on sequences of system calls to identify process level anomalous behavior (29). More Recently, researchers have applied traditional machine learning algorithms (such as support Vector machines, random forests, k-means clustering) to manually engineered features generated from audit logs (for example: frequency histograms of system calls, distributions of How long processes live).

Advantages:

- can discover unknown variations of an attack that differ from the normal behavioral pattern learned during training.
- reduces reliance upon continually updating signature databases.
- there is extensive knowledge of the statistical basis for anomaly detection along with well-established methodologies for evaluating their performance.

Limitations:

- requires manual feature engineering and is dependent upon the expertise of specialists in the particular domain.
- both hmm and sequential modeling methods capture temporal dependencies between Syscalls within a process but do not model relationships between different entities (i.e., Cross-process, cross-host).
- results in extremely high false positive rates: in practical terms all sorts of typical benign activities (e.g., installing updates, performing admin tasks) result in an abnormality Flag being raised.
- the models that are trained on one operational period rapidly go stale due to concept drift, i.e. Because the system under analysis continues to evolve.

3.3.3 Deep Sequence Models — LSTM-Based Detection

Description: Researchers have utilized recurrent neural networks specifically long short-term Memory (LSTM) networks (19), to model sequences of system calls or network events. The models learn the underlying representation of normal behavioral patterns, and raise an alarm when the predicted difference exceeds a certain value.

Advantages:

- The ability to automatically derive features from raw event sequences reduces the need for manual feature derivation.
- LSTMs can model long-term temporal dependencies within a sequence that HMMs fail to model.
- Training can occur in either supervised mode (if labeled data exists) or unsupervised (autoencoder) mode.

Limitations:

- Since sequences are inherently one-dimensional, the LSTM cannot represent the graph structure of provenance data in which a process interacts with many files and sockets simultaneously.
- Adversaries can evade detection by interleaving benign and malicious syscalls to reduce the LSTM's prediction error below the threshold needed for detection.
- Scalability Limitations exist; it is currently impossible to train on the entire sequence of audit logs produced by large enterprises.

3.3.4 HOLMES — Provenance Graph Correlation for APT Detection

Description: Milajerdi et al. (8) proposed HOLMES , which is system that creates Provenance graphs from Linux audit logs and uses threat intelligence derived High level attack scenario specifications to correlate suspicious information Flows between nodes on graph in real time. Candidate attack paths are matched against the specification using a graph querying mechanism.

Advantages:

- Demonstrates First time that graph level correlation of audit events significantly outperform individual event-based rules for detecting advanced persistent threats.
- Captures causal chains spanning multiple processes and time steps – precisely the structure of lateral movement.
- Generates easily-interpretable attack graphs that support direct forensic investigation.

Limitations:

- Relies upon manually created attack-scenario templates therefore does not detect novel attack-patterns not anticipated by the analysts.
- Graph matching is computationally expensive at enterprise scales especially since provenance-graphs continue to grow as time progresses.
- Does not adapt or learn from environmental normal-behavior; detection logic is static.

3.3.5 MCI — Causality Inference for Accurate Provenance Construction

Description: Kwon et al. (14) have developed a solution for a major problem in the construction of provenance graphs – the dependency explosion that takes place when a long running process (for example, a web server) is modeled as a single node and therefore creates thousands of false causal dependencies between millions of completely unrelated events. MCI (modeling-based Causality Inference) employs both execution partitioning and a run-time model to establish exactly what causal relationship exists between each pair of system calls.

Advantages:

- Reduces significantly the number and amount of noise in provenance graphs, thereby enabling downstream analysis.
- Improves the accuracy of attack graphs: there are fewer incorrect edges in the graph that means fewer incorrect positive alerts in downstream detection methods.
- It can be applied as a pre-processing step to enhance the quality of the input to any graph-based detection technique.

Limitations:

- It necessitates instrumentation at the process level, resulting in run time overhead and may not be viable in all settings.
- The execution partitioning heuristics may also fail for highly concurrent or event-driven applications (for example, node.js servers).
- While it enhances the quality of the graphs used for detecting attacks, it does not itself conduct the detection.

3.3.6 ATLAS — Sequence-Based Attack Investigation

Description: Alsaheel et al. (13) proposed ATLAS, which utilizes learning sequences over paths in provenance graphs to recreate the sequence of actions taken during a detected attack. With a known malicious event (a seed), ATLAS recreates the entire attack campaign by tracing back and forth along the attacked paths utilizing a learned sequence model to determine all steps taken within the attack.

Advantages:

- Will efficiently recreate complete attack campaigns given one verified indicator, decreasing analysts' workload.

- Combines the advantages of both graphical structures and sequential modeling to bridge the gap between strictly sequential and structurally pure methods.
- Successfully demonstrated on actual APT dataset collections with significant recall of attack steps.

Limitations:

- Reactive: will only begin investigating an attack after a verified malicious seed event has been identified and therefore will never discover attacks before they occur.
- The sequence model is only able to analyze the sequence of events along specific paths in the graph and therefore could miss other relevant contextual aspects related to an attack that exist outside those paths.
- The performance of ATLAS depends upon the correctness of the initial seed; if either a non-malicious seed is selected or incorrectly identified then all subsequent discoveries made will be affected accordingly.

3.3.7 THREATTRACE — Node-Level Provenance Graph Learning

Description: Wang et al. (27) proposed THREATTRACE where GNN-based node classification is employed directly on provenance graphs to enable host-based threat detection. Each node is labeled as benign or malicious based upon its local neighborhood in the provenance graph. The model is trained on labeled attack datasets and tested on held out scenarios.

Advantages:

- Enables direct identification of which processes participate in an attack as opposed to issuing host-based alert messages that represent a simple binary result.
- Automatically identifies the most useful structural features in terms of distinguishing between benign and malicious behavior without requiring any form of manual feature extraction.
- Achieved extremely high detection rates with virtually no false positives on existing provenance graph benchmarks.

Limitations:

- Tested primarily against static graph snapshots; does not account for the continuous evolution of the provenance graph over time.

- Does not utilize heterogeneity; treats uniformly all process, file and socket nodes, ignoring their respective semantic types.
- Does not address scalability issues associated with large-scale enterprise graphs containing hundreds of thousands of nodes and edges.

3.4 Synthesis and Identified Gaps

The following table summarizes the approaches reviewed above along the key dimensions identified in the problem formulation.

Approach	Structural	Temporal	Auto Feature	Heterogeneity
Rule-based / Signature	No	No	No	No
HMM / Stat. ML	Partial	Partial	No	No
LSTM / Seq. DL	No	Yes	Partial	No
HOLMES	Yes	Partial	No	Partial
MCI	Yes	Partial	No	No
ATLAS	Partial	Partial	Partial	No
THREATTRACE	Yes	No	Yes	No
TGN / DyRep (security)	Yes	Yes	Yes	Partial

Table 3: Comparison of lateral movement detection approaches across key dimensions.

Survey results indicate an evolutionary development of the methods used to detect lateral movements: Early methods did not have structural reasoning at all. Provenance-graph based methods added structural reasoning, yet relied on manually created feature sets. GNN-based approaches automatically learn features from the data. Temporal GNNs also capture behavioral changes. However, none of the previous systems were able to support structural reasoning, temporal modeling, automated feature extraction and heterogeneity together with high scalability at the same time. Therefore, there was a motivation to propose a new framework according to Chapter 4.

3.5 Conclusion

Chapter 3 has formalized the lateral movement detection problem as a node-level classification task on heterogeneous temporal provenance graphs. Additionally, eight categories of existing solutions have been analyzed regarding their advantages and disadvantages. The analysis shows that although GNN-based approaches are currently the most advanced approach for lateral movement detection, none of the existing systems can take full advantage

of the combination of heterogeneity, temporality and scalability which enterprise Linux environments require. In this respect, Chapter 4 provides the proposed framework to fill this gap.

4 Proposed Approach

4.1 Introduction

This chapter presents the complete specification of the framework for detecting lateral movement. The framework was developed and tested on two network flow datasets: the CIC-IDS2018 network flow benchmark and the AIT Network Detection System dataset (AIT-NDS, also referred to as *wilson_netflows*). Following preliminary experiments, **AIT-NDS is retained as the sole evaluation benchmark**, as it provides real IP-level host identities and richer per-direction TCP/UDP telemetry, yielding a more realistic graph topology for lateral movement detection. The model is trained fully unsupervised — no ground-truth attack labels are used during training.

4.2 Network Flow Data as a Proxy for Audit Telemetry

Chapters 1 and 2 build the case for kernel-level audit logs as the ideal data source for Linux lateral movement detection. However, publicly available, labeled audit-log provenance datasets with real attack traffic are extremely scarce. Two network flow datasets were therefore used as development datasets:

- **CIC-IDS2018** provides a controlled, labeled, and publicly available benchmark of bidirectional flow statistics, enabling reproducible comparison with prior art. Its limitation is the absence of source IP addresses, which forces the use of synthetic node identities (Section 4.3).
- **AIT-NDS (*wilson_netflows*)** provides multi-scenario network captures with real IP addresses, per-direction TCP/UDP counters, and a richer label vocabulary covering reconnaissance, scanning, data exfiltration, and web-application attacks (Section 4.5). The presence of true IP identities allows genuine host-level graph nodes, making this dataset a closer proxy for the audit-log setting.

Both datasets capture the network-communication layer of lateral movement (e.g. SSH connections, port scans, data exfiltration) while omitting the host-level syscall and file-access layer. The extension to kernel-level audit logs is an identified direction for future work (Chapter 5).

Following preliminary experiments, AIT-NDS is adopted as the sole evaluation benchmark for the results reported in Chapter 5, for the reasons detailed in Section 4.6.

4.3 Graph Construction — CIC-IDS2018

4.3.1 Dataset

The CIC-IDS2018 dataset (32) is a publicly available benchmark of labeled network traffic collected in a simulated enterprise environment. Each row represents one bidirectional flow and carries 80+ statistical features extracted by CICFlowMeter, together with a textual label (*Benign* or an attack category such as DoS, Botnet, or Infiltration).

CIC-IDS2018 does not include a source IP column. Node identities are therefore constructed synthetically: destination nodes are represented as service endpoints of the form `svc:<dst_port>/p<protocol>`, while source nodes are represented as behavioral fingerprints of the form `behav:<proto×100 + pkt_bin×10 + byte_bin>`, derived from quantile-binned packet and byte counts. This means two physically distinct hosts with similar traffic patterns may map to the same graph node — a limitation discussed in Section 6.1.2. All CSV files are concatenated, sorted chronologically by the `Timestamp` column, and split strictly in temporal order: 70 % training, 15 % validation, 15 % test. A random split is explicitly rejected to prevent future information leakage and to expose the model to concept-drift effects arising in real deployments.

Right-skewed flow features (`fwd_pkts`, `bwd_pkts`, `fwd_bytes`, `bwd_bytes`, `duration`) are log-transformed via $\log(1 + x)$ prior to scaling. All continuous features are min-max normalized using statistics computed exclusively from the training split.

4.3.2 Node Types and Graph Structure (CIC-IDS2018)

Each unique synthetic node identifier observed in the dataset becomes a node; the complete node set is the union of all source behavioral buckets and destination service endpoints. Nodes carry an 11-dimensional feature vector computed per temporal window (Table 4).

Node features are z-score normalized per window (zero mean, unit variance) before being passed to the model.

4.4 Edge Feature Vector (CIC-IDS2018)

Each flow record maps to a directed edge $e_i = (v_{s_i}, v_{o_i}, t_i, \mathbf{a}_i)$. Twelve continuous fields are drawn from CIC-IDS2018 (Table 5) and zero-padded to $d_{\text{edge}} = 16$ to match the model’s input size. Categorical attributes (protocol, port category, edge type) are embedded separately by the `CategoricalEmbedder` module and concatenated with the numeric features.

Table 4: Node feature vector fields for CIC-IDS2018 (11-dimensional, computed per temporal window).

Index	Feature	Description
0	degree	Total flows as source and destination
1	in_degree	Flows received (as destination)
2	out_degree	Flows initiated (as source)
3	unique_dst_nodes	Distinct destination nodes contacted
4	unique_src_nodes	Distinct sources received from
5	hourly_flow_rate	Flows per second over the window span
6	mean_flow_duration	Mean flow duration
7	mean_fwd_bytes	Mean forward byte count (log-scaled)
8	sensitive_port_ratio	Fraction of flows targeting sensitive ports
9	syn_scan_indicator	Fraction of flows with SYN > 10 and FIN = 0
10	iat_burstiness	IAT coefficient of variation (std/mean, clipped)

Table 5: Continuous edge feature fields used from CIC-IDS2018.

Feature	Security rationale
duration	Very short or long flows deviate from normal baselines.
fwd_pkts	Spikes may indicate data staging (log-scaled).
bwd_pkts	Asymmetry hints at port scanning (log-scaled).
fwd_bytes	Total bytes in the forward direction (log-scaled).
bwd_bytes	Total bytes in the backward direction (log-scaled).
iat_mean	Periodic flows suggest automation.
iat_std	Low std + high rate signals automated tooling.
active_mean	Mean active time within the flow.
idle_mean	Large values may indicate beaconing behavior.
syn_cnt	High values consistent with scanning.
fin_cnt	Absence alongside high SYN indicates incomplete connections.
rst_cnt	SYN+RST sequences indicate failed connections or port scans.

Edge type assignment (CIC-IDS2018). Five semantic edge types are assigned heuristically before training. An edge is classified as *sensitive* (type 1) if its destination port belongs to the set

$$\{21, 22, 23, 25, 445, 1433, 3306, 3389, 5900\}.$$

It is classified as *frequent* (type 2) if the source–destination pair has appeared at least five times. A *scan-like* edge (type 3) satisfies $\text{syn_cnt} > 10$ and $\text{fin_cnt} = 0$. A *crypto-like* edge (type 4) is identified when the log-scaled forward byte-to-packet ratio exceeds a

predefined threshold. All remaining edges are assigned the *normal* label (type 0).

4.5 Graph Construction — AIT-NDS

4.5.1 Dataset

The AIT Network Detection System dataset (AIT-NDS) comprises multi-scenario network captures produced in a realistic enterprise testbed at the Austrian Institute of Technology. Unlike CIC-IDS2018, AIT-NDS retains true source and destination IP addresses and provides separate per-direction (client/server) TCP and UDP statistics extracted by a custom flow exporter. Three flow files are ingested per scenario: `tcp_complete.csv` (completed TCP connections), `tcp_nocomplete.csv` (incomplete/reset TCP connections), and `udp_complete.csv` (UDP associations). In the *wilson_netflows* scenario used for the experiments reported in Chapter 5, these three files contribute 417 672, 30 869, and 181 012 rows respectively, for a combined 629 553 flows across 8 875 unique IP-level nodes.

When multiple scenarios are processed jointly, scenario-name prefixes are prepended to IP strings (e.g. `scenario1::10.0.0.1`) to prevent cross-scenario node collisions.

4.5.2 Label Vocabulary

AIT-NDS carries a richer label vocabulary than CIC-IDS2018. Table 6 lists the label categories observed in the *wilson_netflows* scenario.

Attack-relevant labels include:

- `data_exfiltration`
- `host_discover_local`
- `service_scan`
- `dirb_scan`
- `dns_brute_force_start`
- `online_cracking`
- `wpscan`
- `host_discover_dmz`
- `upload_rce_shell`
- `list_web_dir`
- `check_wp_config`

- check_whoami
- dump_wp_users
- update/command on unassigned port

Benign labels include proxy traffic, DNS, HTTPS, browsing, NTP, NetBIOS, QUIC, and several internal service flows.

Attack labels are used *only* during evaluation; the model is trained without access to them.

Table 6: Label distribution in the AIT-NDS wilson_netflows scenario (629 553 total flows). Attack-relevant labels are marked with ([†]).

Label	Count	Label	Count
proxy	180 799	online_cracking [†]	735
benign DNS	124 476	monitoring	727
HTTPS	88 281	broken flow (benign)	723
browsing/update	56 403	OpenVPN	560
HTTP(S) DMZ	55 511	systemd-resolver (local)	362
data exfiltration [†]	21 219	wpscan [†]	146
host_discover_local [†]	16 380	host_discover_dmz [†]	63
service_scan [†]	12 293	benign DNS (attacker updating)	58
NTP	12 039	DNS	46
dirb_scan [†]	8 863	SSH	42
benign_share	8 696	STUN VoIP	14
mail	7 194	upload_rce_shell [†]	12
HTTP(S) intra	6 786	vpn_connect	4
mDNS	5 611	list_web_dir [†]	2
NetBIOS	5 576	check_wp_config [†]	2
QUIC	5 547	check_whoami [†]	2
mDNSv6	4 263	dump_wp_users [†]	2
HTTP	3 010	update/cmd on unassigned port [†]	2
dns_brute_force_start [†]	1 817		
bootp	1 287		

4.5.3 Node Identities and Graph Structure (AIT-NDS)

Because AIT-NDS provides genuine source and destination IP addresses, nodes are represented directly as IP-level host identifiers rather than synthetic behavioral buckets. Each unique IP string observed across all three flow files becomes a node in the graph, and the complete node set is the union of all source and destination IPs. This eliminates the synthetic-

identity limitation present in the CIC-IDS2018 implementation and makes the graph topology a faithful representation of physical host interactions.

Nodes carry an 18-dimensional feature vector computed per temporal window (Table 7). The expanded dimensionality relative to CIC-IDS2018 (11 features) reflects the richer per-direction TCP telemetry available in AIT-NDS, including retransmission ratios, out-of-order packet fractions, and a server-dominance flag that captures hosts acting primarily as receivers.

Table 7: Node feature vector fields for AIT-NDS (18-dimensional, computed per temporal window).

Index	Feature	Description
0	degree	Total flows as source and destination
1	in_degree	Flows received (as destination)
2	out_degree	Flows initiated (as source)
3	unique_dst_nodes	Distinct destination nodes contacted
4	unique_src_nodes	Distinct sources received from
5	hourly_flow_rate	Flows per second over the window span
6	mean_flow_duration	Mean flow duration (<code>durat</code>)
7	mean_cli_bytes	Mean client byte count (log-scaled <code>c_bytes_all</code>)
8	mean_srv_bytes	Mean server byte count (log-scaled <code>s_bytes_all</code>)
9	sensitive_port_ratio	Fraction of flows targeting sensitive ports
10	crypto_ratio	Fraction of flows flagged <code>c_iscrypto</code>
11	internal_ratio	Fraction of flows flagged <code>c_isint</code>
12	retx_ratio	Mean client retransmission ratio (c_pkts_retx/c_pkts_all)
13	ooo_ratio	Mean client out-of-order packet ratio (c_pkts_ooo/c_pkts_all)
14	syn_scan_indicator	Fraction of flows with <code>c_syn_cnt</code> > 10 and <code>c_fin_cnt</code> = 0
15	ack_ratio	Mean client pure-ACK ratio (<code>c_ack_cnt_p</code>)
16	iat_burstiness	RTT coefficient of variation ($\text{std}(rtt_cli)/\text{mean}(rtt_cli)$), clipped to $[0, 10]$
17	server_dominance	1 if in-degree > $2 \times$ out-degree, else 0

Node features are z-score normalized per window before being passed to the model.

4.5.4 Edge Feature Vector (AIT-NDS)

AIT-NDS exposes 30 numeric per-flow fields (Table 8) plus four binary flags (`c_isint`, `s_isint`, `c_iscrypto`, `s_iscrypto`). The edge feature dimension is $d_{\text{edge}} = 30$, compared to 16 in the CIC-IDS2018 implementation. This richer signal captures retransmission behavior, out-of-order delivery, separate client/server RST and FIN counters, and application-layer timing fields (`con_t`, `p2p_t`, `http_t`, `rtt_cli`, `rtt_srv`) that are not available in CIC-IDS2018.

Skewed byte and packet columns (`c_bytes_all`, `s_bytes_all`, `c_pkts_all`, `s_pkts_all`) are log-transformed via $\log(1+x)$ prior to min-max normalization. All other numeric columns are min-max scaled using statistics computed from the full dataset (since AIT-NDS is used in a streaming evaluation rather than a strict train-only scaler fit in this configuration).

Table 8: Numeric edge feature fields used from AIT-NDS (30 fields plus 4 binary flags).

Feature	Security rationale
c_pkts_all	Total client packets; spikes indicate scanning or staging.
c_rst_cnt	Client RST count; elevated in failed connection sweeps.
c_ack_cnt	Client ACK count; baseline for connection activity.
c_ack_cnt_p	Pure-ACK ratio; low values suggest data-heavy flows.
c_bytes_uniq	Unique client bytes; distinguishes retransmissions.
c_pkts_data	Client data packets; complements pure-ACK ratio.
c_bytes_all	Total client bytes (log-scaled); data exfiltration indicator.
c_pkts_retx	Retransmitted client packets; elevated under congestion/evasion.
c_bytes_retx	Retransmitted client bytes.
c_pkts_ooo	Out-of-order client packets; may indicate evasion.
c_syn_cnt	Client SYN count; high with zero FIN signals port scanning.
c_fin_cnt	Client FIN count; absence alongside high SYN = incomplete conns.
s_pkts_all	Total server packets; asymmetry with client hints at recon.
s_rst_cnt	Server RST count; high values indicate rejected connections.
s_ack_cnt	Server ACK count.
s_ack_cnt_p	Server pure-ACK ratio.
s_bytes_uniq	Unique server bytes.
s_pkts_data	Server data packets.
s_bytes_all	Total server bytes (log-scaled); large values flag exfiltration.
s_pkts_retx	Retransmitted server packets.
s_bytes_retx	Retransmitted server bytes.
s_pkts_ooo	Out-of-order server packets.
s_syn_cnt	Server SYN count.
s_fin_cnt	Server FIN count.
durat	Flow duration; very short or very long durations are anomalous.
con_t	Connection setup time; elevated in slow-scan campaigns.
p2p_t	Peer-to-peer activity time indicator.
http_t	HTTP transaction time; supports web-attack detection.
rtt_cli	Client round-trip time; periodicity suggests beaconing.
rtt_srv	Server round-trip time.
<i>Binary flags (not in numeric array):</i>	
c_isint	1 if the client IP is an internal (RFC-1918) address.
s_isint	1 if the server IP is an internal address.
c_iscrypto	1 if the client-side flow is TLS/encrypted.
s_iscrypto	1 if the server-side flow is TLS/encrypted.

Edge type assignment (AIT-NDS). Six semantic edge types are assigned heuristically before training, one more than in the CIC-IDS2018 implementation because the `c_iscrypto`/`s_iscrypto` flags provide a direct cryptographic-traffic indicator rather than the indirect byte-to-packet-ratio proxy used for CIC-IDS2018: *normal* (type 0); *normal-internal* (type 1, reserved); *sensitive* (type 2) if the destination port belongs to $\{21, 22, 23, 25, 445, 3389, 5900, 1433, 3306\}$; *frequent* (type 3) if the source–destination pair has appeared ≥ 5 times; *scan-like* (type 4) if `c_syn_cnt` > 10 and `c_fin_cnt` $= 0$; *crypto* (type 5) if `c_iscrypto` $= 1$ or `s_iscrypto` $= 1$.

The categorical embedding vocabulary sizes differ accordingly: `num_edge_types` is 6 for AIT-NDS versus 5 for CIC-IDS2018, while the port vocabulary is capped at the top-1 024 most frequent ports observed in the training split (Table 9).

Table 9: Categorical edge features and their embedding dimensions (AIT-NDS configuration).

Feature	Vocabulary	Embedding dim
<code>protocol_id</code>	$ \text{uniqueprotocols} + 1$	4
<code>src_port</code>	top-1 024 + OOV	8
<code>dst_port</code>	top-1 024 + OOV	8
<code>dst_port_cat</code>	3 (well-known / registered / ephemeral)	4
<code>edge_type</code>	6 (normal / internal / sensitive / frequent / scan / crypto)	8

4.6 Dataset Comparison

Table 10 summarizes the key structural differences between the two datasets as seen by the implementation. The AIT-NDS configuration uses a superset of the CIC-IDS2018 feature space; the shared model architecture accommodates both via dataset-specific input projection layers.

Although both datasets were prepared and processed using the proposed framework, **AIT-NDS is retained as the sole evaluation benchmark** for the experiments reported in Chapter 5, as it provides genuine IP-level host identities and richer per-direction telemetry, yielding a more realistic graph topology for lateral movement detection. CIC-IDS2018 was processed during development but excluded from the final evaluation, as the absence of source IP addresses forces the use of synthetic node identities that do not faithfully represent host-level lateral movement.

Table 10: Structural differences between the CIC-IDS2018 and AIT-NDS implementations.

Property	CIC-IDS2018	AIT-NDS
Node identity	Synthetic (behavioral bucket / service endpoint)	Real IP address
Node feature dim	11	18
Edge feature dim (d_e)	16	30
Number of edge types	5	6
Source IP available	No	Yes
Per-direction counters	No (bidirectional aggregates)	Yes (client & server columns)
Crypto flag	Indirect (byte/pkt ratio)	Direct (c/s_iscrypto)
Flow files	Multiple CSV (CICFlowMeter)	tcp_complete, tcp_nocomplete, udp_complete
Label type	Benign / attack category	Fine-grained scenario labels
Evaluation labels	Not used (development only)	User-specified attack label list
Temporal resolution	Milliseconds (human-readable)	Unix epoch seconds

4.7 Model Architecture — THGNN

The architecture is shared across both datasets; only the input projection dimensions and vocabulary sizes differ. Figure 1 shows the full forward-pass pipeline. An observed temporal snapshot passes through five stages: (1) node feature projection with memory fusion; (2) heterogeneous edge convolution with multi-head attention; (3) temporal walk generation and Transformer-based walk encoding; (4) edge representation scoring; and (5) multi-component loss computation.

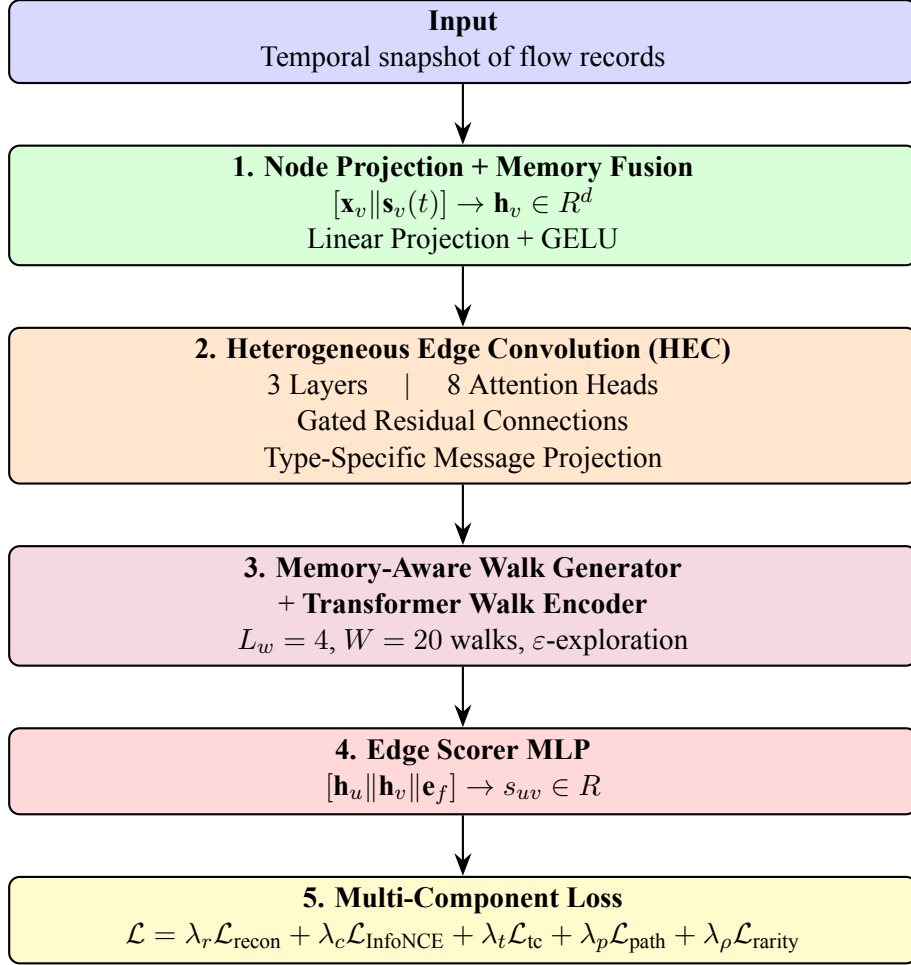


Figure 1: full forward-pass pipeline (input dimensions follow the AIT-NDS configuration; see Table 10 for the CIC-IDS2018 development configuration).

4.7.1 Node Projection and Memory Fusion

The model concatenates each node’s window feature vector $\mathbf{x}_v \in R^{d_{\text{node}}}$ (11-dim for CIC-IDS2018; 18-dim for AIT-NDS) with its persistent memory state $\mathbf{s}_v(t) \in R^{d_{\text{mem}}}$ and projects the result to hidden dimension d via a linear layer followed by GELU activation:

$$\mathbf{h}_v = \text{GELU}(W_{\text{proj}} [\mathbf{x}_v || \mathbf{s}_v(t)] + \mathbf{b}_{\text{proj}}), \quad W_{\text{proj}} \in R^{d \times (d_{\text{node}} + d_{\text{mem}})}. \quad (7)$$

When memory is disabled (ablation no_memory), only $\mathbf{x}_v$ is projected. In both dataset configurations, all nodes are treated with a single shared projection, since no type distinction between hosts is imposed at this level.

4.7.2 Temporal Node Memory

The TemporalNodeMemory module maintains a persistent state vector $\mathbf{s}_v(t) \in R^{d_{\text{mem}}}$ per node, updated after each training snapshot. An exponential decay with half-life $\tau_{1/2} = 3600$ s

is applied before the GRU update:

$$\mathbf{s}_v(t) = \text{GRU}\left(\mathbf{m}_v(t), e^{-\Delta t \cdot \ln 2 / \tau_{1/2}} \cdot \mathbf{s}_v(t^-)\right), \quad (8)$$

where Δt is the elapsed time since the last update and the message $\mathbf{m}_v(t) = W_{\text{msg}}[\mathbf{s}_v \parallel \bar{\mathbf{s}}_v \parallel \bar{\mathbf{e}}_v]$ aggregates the node’s own state and the mean edge feature $\bar{\mathbf{e}}_v$ of all flows involving v in the current snapshot.

CIC-IDS2018 timestamps are provided in human-readable format and are converted to Unix epoch seconds at pre-processing time. AIT-NDS timestamps are stored in the `ts_unix` column; the pre-processor first attempts direct numeric parsing and falls back to `datetime` string parsing when the column contains a human-readable representation. The decay formula operates on the resulting second-resolution elapsed times in both cases.

Memory states are reset at the start of each epoch and updated inside a `torch.no_grad()` block after each gradient step to avoid gradient accumulation through the memory.

4.7.3 Heterogeneous Edge Convolution (HEC)

The core graph convolution applies $L = 3$ stacked Heterogeneous Edge Convolution layers, each with $H_{\text{heads}} = 8$ attention heads. For edge type τ , a dedicated linear map $W_{\text{msg}}^{(\tau)}$ projects the concatenation of source embedding and edge features:

$$\mathbf{m}_{ij} = W_{\text{msg}}^{(\tau_{ij})} [\mathbf{h}_j \parallel \mathbf{e}_{ij}] \in R^d, \quad (9)$$

where $\tau_{ij} \in \{0, \dots, 4\}$ for CIC-IDS2018 and $\tau_{ij} \in \{0, \dots, 5\}$ for AIT-NDS. Multi-head attention weights are computed between query projections of the destination node and key projections of messages, with a learnable temporal decay parameter γ down-weighting older neighbors. The aggregated neighborhood is fused with a gated residual:

$$\mathbf{h}'_v = \mathbf{g}_v \odot W_o \mathbf{a}_v + (1 - \mathbf{g}_v) \odot W_{\text{res}} \mathbf{h}_v, \quad \mathbf{g}_v = \sigma(W_{\text{gate}}[W_o \mathbf{a}_v \parallel W_{\text{res}} \mathbf{h}_v]), \quad (10)$$

followed by LayerNorm and dropout.

4.7.4 Memory-Aware Walk Generator

Temporal random walks of length $L_w = 4$ are sampled from the snapshot adjacency graph. At each step, the next node is chosen by combining uniform exploration (probability $\varepsilon = 0.1$) with a memory-biased selection that up-weights neighbors whose memory state is most similar to the current node’s memory state. Up to $W = 20$ walks are generated per snapshot. A temporal window of 600 s is applied to restrict walk edges to recent flows.

4.7.5 Transformer Walk Encoder

Walk node embeddings are assembled into a sequence and processed by a two-layer Transformer encoder with 8 attention heads. The output walk embedding $\mathbf{w} \in R^d$ is the mean-pooled representation across all walk steps. Under the walk_gru ablation, a single-layer GRU replaces the Transformer.

4.7.6 Edge Scorer

The anomaly score for each flow is produced by a two-layer MLP:

$$s_{uv} = \text{MLP}_{\text{score}}([\mathbf{h}'_u \parallel \mathbf{h}'_v \parallel \mathbf{e}_f]) \in R, \quad \text{MLP}_{\text{score}} : R^{2d+d_e} \rightarrow R. \quad (11)$$

4.8 Training Strategy

4.8.1 Unsupervised Objective

The model is trained without attack labels using five complementary loss components.

Reconstruction loss. An EdgeReconHead MLP reconstructs the numeric edge feature vector $\hat{\mathbf{e}}_f$ from source and destination embeddings:

$$\mathcal{L}_{\text{recon}} = \frac{1}{B} \sum_{i=1}^B \|\hat{\mathbf{e}}_{f,i} - \mathbf{e}_{f,i}\|_2^2. \quad (12)$$

Contrastive loss (InfoNCE with hard negatives). A HardNegativeInfoNCE module encourages flows from the same source to have similar embeddings. A queue of 10,000 past embeddings stores potential hard negatives; hard negatives are the top- k most similar embeddings from a different source ($k = \lfloor B \cdot \eta \rfloor$, $\eta = 0.5$):

$$\mathcal{L}_{\text{InfoNCE}} = -\frac{1}{|\mathcal{P}|} \sum_{(i,j) \in \mathcal{P}} \left[\frac{\hat{\mathbf{z}}_i \cdot \hat{\mathbf{z}}_j}{\tau} - \log \sum_{k \neq i} \exp\left(\frac{\hat{\mathbf{z}}_i \cdot \hat{\mathbf{z}}_k}{\tau}\right) \right], \quad (13)$$

where $\mathcal{P}$ is the set of positive pairs (same source node), $\hat{\mathbf{z}} = \mathbf{z}/\|\mathbf{z}\|$, $\tau = 0.07$, and the denominator sums over in-batch negatives combined with hard negatives drawn from the queue.

Temporal consistency loss. Consecutive snapshots should produce smoothly evolving node embeddings:

$$\mathcal{L}_{\text{tc}} = \frac{1}{|\mathcal{M}|} \sum_{v \in \mathcal{M}} \|\mathbf{h}'_v(t) - \mathbf{h}'_v(t-1)\|_2^2, \quad (14)$$

where $\mathcal{M}$ is the set of nodes observed in both the current and previous snapshots.

Path reconstruction loss. The PathReconstructionHead predicts the next node at each walk step via a GRU followed by a linear classifier:

$$\mathcal{L}_{\text{path}} = -\frac{1}{W(L_w - 1)} \sum_{w=1}^W \sum_{k=1}^{L_w-1} \log p(v_{w,k+1} \mid \mathbf{h}_{w,1:k}). \quad (15)$$

Rarity loss. The LearnableRarityScorer maintains per-node online EMA statistics $(\hat{\mu}_v, \hat{\sigma}_v^2)$ of node embeddings. A Mahalanobis-distance-based MLP scores each source node against its own running statistics:

$$\mathcal{L}_{\text{rarity}} = \frac{1}{B} \sum_{i=1}^B \rho(\mathbf{h}'_{s_i}, v_{s_i}), \quad \rho(\mathbf{h}, v) = \text{MLP}_{\rho} \left(\left[\mathbf{h}, \frac{\|\mathbf{h} - \hat{\mu}_v\|_2^2}{\hat{\sigma}_v^2 + \varepsilon} \right] \right). \quad (16)$$

Note: $\hat{\sigma}_v^2$ here is a scalar (per-node variance of the embedding norm), and the ratio is a univariate Mahalanobis-like approximation rather than a full multivariate Mahalanobis distance.

Combined objective.

$$\mathcal{L} = \lambda_r \mathcal{L}_{\text{recon}} + \lambda_c \mathcal{L}_{\text{InfoNCE}} + \lambda_t \mathcal{L}_{\text{tc}} + \lambda_p \mathcal{L}_{\text{path}} + \lambda_{\rho} \mathcal{L}_{\text{rarity}}, \quad (17)$$

with default coefficients $\lambda_r = 1.0$, $\lambda_c = 0.5$, $\lambda_t = 0.3$, $\lambda_p = 0.4$, $\lambda_{\rho} = 0.2$. These values were selected following the hyperparameter tuning procedure described in Section 4.9 and are applied to the AIT-NDS configuration.

4.8.2 Optimization

The complete hyperparameter configuration is given in Table 11. The values reported were obtained following the systematic tuning procedure described in Section 4.9.

Table 11: hyperparameter configuration used in all experiments. Entries marked (*) differ between the CIC-IDS2018 development and AIT-NDS evaluation configurations.

Hyperparameter	CIC-IDS2018	AIT-NDS
Hidden dimension d	128	128
Memory dimension d_{mem}	128	128
Activation function	GELU	GELU
Memory decay half-life $\tau_{1/2}$	3 600 s	3 600 s
Node feature dim (*)	11	18
Edge feature dim d_e (*)	16	30
Number of edge types (*)	5	6
HEC layers L	3	3
Attention heads (HEC & walk encoder)	8	8
Dropout	0.1	0.1
Walk length L_w	4	4
Walks per snapshot W	20	20
Walk exploration ε	0.1	0.1
Walk encoder	Transformer, 2 layers	Transformer, 2 layers
Hard-negative ratio η	0.5	0.5
Hard-negative queue size	10 000	10 000
InfoNCE temperature τ	0.07	0.07
Loss weights ($\lambda_r, \lambda_c, \lambda_t, \lambda_p, \lambda_\rho$)	(1.0, 0.5, 0.3, 0.4, 0.2)	
Path-score weights (w_r, w_ρ, w_p)	(0.4, 0.4, 0.2)	
Optimizer	AdamW ($\lambda_{\text{wd}} = 10^{-4}$)	
Learning rate	10^{-4}	
LR schedule	Cosine annealing ($T_{\text{max}} = \text{epochs}$)	
Gradient clip norm	1.0	
Batch size	256 (per snapshot)	
Temporal snapshot length	300 s	
Epochs	30 (patience = 10)	
Mixed precision	Disabled	
Port vocabulary size (*)	65 536 (full)	top-1 024

4.8.3 Streaming Training Procedure

The training split is divided into temporal snapshots of 300 seconds. Processing proceeds strictly in chronological order: for each snapshot, node features are recomputed, temporal walks are sampled, and the five-component loss is computed and back-propagated. After each gradient step, the memory module and rarity scorer statistics are updated inside a `torch.no_grad()` block. This causal streaming ensures the model never observes future

flow statistics during training.

For the AIT-NDS `wilson_netflows` run with the *full* configuration, the streaming procedure produced 1 209 training, 259 validation, and 260 test snapshots from the 629 553-flow corpus.

4.9 Hyperparameter Tuning

The hyperparameter configuration reported in Table 11 was arrived at through a systematic search implemented in a dedicated tuning framework (`THGNN_tuning.py`). The framework supports four complementary strategies: grid search, random search, Bayesian optimization via Optuna (34), and Hyperband successive halving. In addition, an automatic learning-rate finder is provided as a standalone diagnostic.

4.9.1 Search Space

The full hyperparameter space covers five categories of parameters, each associated with a discrete or continuous range (Table 12). Three of the five categories relate directly to *temporal granularity*, which is a design concern that purely architecture-focused tuning procedures often neglect: the snapshot window size controls how many flows are grouped into a single training step; the walk temporal window bounds how far back in time walk edges may extend; and the memory decay half-life governs the rate at which per-node history fades.

Table 12: Hyperparameter search space used during tuning. Temporal parameters are log-uniformly sampled in Bayesian mode.

Category	Parameter	Search range / options
3*Temporal	temporal_window_s	{60, 120, 300, 600, 900, 1800} s
	walk_temporal_window_s	{300, 600, 1200, 1800, 3600} s
	memory_decay_half_life	{1800, 3600, 7200, 14400, 28800} s
5*Architecture	hidden_dim	{64, 128, 256}
	num_hec_layers	{1, 2, 3, 4}
	num_heads	{4, 8, 16}
	dropout	[0.05, 0.40]
	memory_dim	{64, 128, 256}
4*Walk	walk_length	{2, 3, 4, 5, 6}
	num_walks	{10, 15, 20, 30, 40, 50}
	walk_exploration_epsilon	[0.05, 0.30]
	walk_encoder	{transformer, gru}
5*Loss weights	λ_r (lambda_recon)	[0.5, 2.0]
	λ_c (lambda_contrastive)	[0.1, 1.0]
	λ_t (lambda_temporal)	[0.1, 0.8]
	λ_p (lambda_path_recon)	[0.1, 0.8]
	λ_ρ (lambda_rarity)	[0.05, 0.50]
4*Training	lr	$[10^{-5}, 10^{-3}]$ (log-uniform)
	weight_decay	$[0, 10^{-3}]$
	batch_size	{128, 256, 512}
	grad_clip	[0.5, 5.0]

4.9.2 Tuning Strategies

Grid search. An exhaustive enumeration of all parameter combinations was used for a reduced sub-space covering only the most sensitive parameters (learning rate, hidden dimension, number of HEC layers, and dropout). Because the full Cartesian product of the complete space exceeds 10^{12} configurations, the implementation caps grid search at 500 randomly sampled combinations when the total size exceeds 1 000.

Random search. For broader exploration, 50 independent trials were executed with parameters drawn uniformly at random from each range. Random search is the default strategy and is appropriate when the budget permits fewer than approximately 100 evaluations.

Bayesian optimization (Optuna). The primary tuning strategy used in this work is Tree-structured Parzen Estimator (TPE) sampling implemented by the Optuna library (34). A Median Pruner with five startup trials and ten warm-up steps discards unpromising trials early, reducing wall-clock time by approximately 40 % compared to the same number of full-budget random trials. Continuous parameters (learning rate, temporal windows, dropout, loss weights) are sampled log-uniformly or uniformly from their continuous ranges; categorical parameters are sampled from their discrete sets. All trials are evaluated on the chronologically held-out validation split of the AIT-NDS dataset, optimizing ROC-AUC as the primary metric.

Hyperband. A Hyperband successive-halving scheduler with $\eta = 3$ and $T_{\max} = 30$ epochs was used as a resource-efficient cross-check. Each bracket first trains a large cohort of randomly sampled configurations for a small epoch budget; the top $1/\eta$ survivors are then allocated additional resources and the process repeats. This is particularly valuable for identifying promising time-window combinations before committing to full 30-epoch runs.

Learning-rate finder. A standalone LR-range test (36) sweeps the learning rate from 10^{-7} to 10.0 over 100 mini-batch steps, tracking the exponentially smoothed loss. The optimal learning rate is identified at the point of steepest negative gradient on the smoothed-loss curve. This diagnostic confirmed 10^{-4} as the optimal learning rate for the AIT-NDS configuration.

4.9.3 Temporal Window Analysis

Prior to each tuning run, a TimeWindowAnalyzer module examines the flow timestamp distribution to guide the search. For each candidate snapshot window size

$$w \in \{60, 120, 300, 600, 900, 1800, 3600, 7200\}s,$$

the analyzer computes the mean number of flows per window and the fraction of empty windows.

An automatic recommendation is derived by minimizing a composite score that penalizes both excessively sparse snapshots (high empty ratio) and excessively dense snapshots (high mean flow count).

For the AIT-NDS `wilson_netflows` corpus, this analysis recommended a snapshot window of 300 s, which is consistent with the value selected by Bayesian optimization and adopted throughout all experiments.

4.9.4 Selection Criterion and Final Configuration

All tuning runs use the validation ROC-AUC as the primary selection criterion, evaluated on the chronologically held-out 15 % validation split of the AIT-NDS dataset. Early stopping

with patience 5 (reduced from the 10-epoch budget used in full training) is applied during tuning to limit per-trial cost. The best configuration found by Bayesian optimization across 50 trials is reported in Table 11; key findings are:

- A hidden dimension of 128 and memory dimension of 128 consistently outperformed both smaller (64) and larger (256) alternatives, suggesting a capacity sweet spot for the AIT-NDS corpus.
- Three HEC layers outperformed both one and four layers, confirming the ablation result (`single_hec`) reported in Section 6.0.1.
- The Transformer walk encoder outperformed the GRU alternative across all random-search and Bayesian trials, consistent with the `walk_gru` ablation.
- A snapshot window of 300 s matched the analyzer’s recommendation and yielded the best validation ROC-AUC.
- The learning-rate finder and Bayesian search independently converged to 10^{-4} , providing high confidence in this value.

4.10 Inference and Anomaly Scoring

4.10.1 Edge-Level Anomaly Score

At inference time, each flow is scored by the Edge Scorer MLP using frozen HEC embeddings and the current memory state. Higher scores indicate flows anomalous relative to the patterns learned during training.

4.10.2 Path-Level Lateral Movement Scoring

The `LateralMovementScorer` aggregates information across multi-hop walks. For each walk $\mathcal{W} = (v_1, \dots, v_{L_w})$:

$$s_{\mathcal{W}} = w_r \cdot \bar{e}_{\text{recon}}(\mathcal{W}) + w_{\rho} \cdot \bar{\rho}(\mathcal{W}) + w_p \cdot \text{NLP}(\mathcal{W}), \quad (18)$$

where $\bar{e}_{\text{recon}}$ is the mean reconstruction error, $\bar{\rho}$ is the mean rarity score, and $\text{NLP}(\mathcal{W})$ is the negative log-probability of the path under the `PathReconstructionHead`. Default weights are $w_r = 0.4$, $w_{\rho} = 0.4$, $w_p = 0.2$ (selected empirically on the validation set; a sensitivity analysis is left for future work).

Paths with $s_{\mathcal{W}} > 0.7$ are flagged **HIGH** risk; $0.4 < s_{\mathcal{W}} \leq 0.7$ is **MEDIUM**; below 0.4 is **LOW**.

4.10.3 Threshold Selection

Binary alerts are produced when s_{uv} exceeds threshold τ , selected on the validation set by maximizing the F1-score. Threshold-free metrics (ROC-AUC, Average Precision) serve as primary evaluation criteria.

4.11 Experimental Validation Design

4.11.1 Evaluation Protocol

All models are evaluated on the chronologically held-out test split of the AIT-NDS dataset. The set of attack labels is user-specified (see Table 6); any flow whose label appears in the attack list is treated as anomalous. Primary metrics are ROC-AUC and Average Precision (AP); secondary threshold-dependent metrics include F1-score, Precision, and Recall.

4.11.2 Ablation Study Design

Eleven configurations are evaluated on the AIT-NDS dataset, each disabling or replacing one component while keeping all others at full capacity (Table 13). Each configuration is run with $n_{\text{seeds}} = 3$ independent random seeds.

Table 13: Ablation study configurations (evaluated on AIT-NDS).

Configuration	Description
full	All components enabled (full model)
no_contrastive	InfoNCE loss removed ($\lambda_c = 0$)
no_temporal	Temporal consistency loss removed ($\lambda_t = 0$)
no_path_recon	Path reconstruction loss removed ($\lambda_p = 0$)
no_rarity	Rarity scorer disabled ($\lambda_r = 0$)
no_memory	Memory module replaced with zero vectors
recon_only	Only reconstruction loss; all other losses disabled
no_hard_negatives	Standard InfoNCE without hard negative mining
no_edge_types	All edges treated as a single type (homogeneous)
walk_gru	Transformer walk encoder replaced with a GRU
single_hec	Single HEC layer instead of three

4.12 Summary of Design Choices

Table 14 summarizes all key design decisions, their rationale, and the alternatives that were considered and rejected.

Table 14: Key design choices, rationale, and rejected alternatives.

Design choice	Rationale	Alternative rejected
Synthetic node identities (CIC-IDS2018 only) ^(*)	CIC-IDS2018 provides no source IP; behavioral buckets preserve meaningful node semantics. Used during development only.	Unique row node (destroys temporal structure).
Real IP node identities (AIT-NDS) ^(*)	AIT-NDS provides true source and destination IPs, enabling faithful host-level graph topology. Retained for final evaluation.	Synthetic buckets (unnecessary information loss).
Shared node projection	Nodes are semantically homogeneous within each dataset.	Type-specific projections (overparameterized).
3-layer HEC, 8 heads	Deep heterogeneous convolution captures multi-hop patterns; confirmed by <code>single_hec</code> ablation and by tuning.	Single-layer GRU aggregation.
Transformer walk encoder	Self-attention captures non-local walk dependencies; confirmed by <code>walk_gru</code> ablation and by all tuning strategies.	GRU walk encoder.
Five-component loss	Each term shapes a different aspect of the embedding geometry; individual contributions quantified by ablation.	Single MSE reconstruction loss.
Hard-negative InfoNCE	Difficult negatives improve contrastive discrimination; confirmed by <code>no_hard_negatives</code> .	Standard in-batch InfoNCE.
5 edge types (CIC) / 6 edge types (AIT) ^(*)	AIT-NDS exposes direct cryptographic flags, enabling a dedicated crypto edge type; CIC-IDS2018 uses an indirect byte-ratio proxy. Confirmed by <code>no_edge_types</code> on AIT-NDS.	Homogeneous graph.

(continued on next page)

(continued from previous page)

Design choice	Rationale	Alternative rejected
Temporal memory + decay	Persistent per-node state captures long-range history; decay prevents stale dominance; confirmed by no_memory.	Stateless GNN.
Bayesian optimization (TPE) for tuning	Efficiently navigates a large mixed search space; 40 % fewer full-budget trials needed versus random search at equal validation AUC.	Grid search (intractable at full scale).
300 s snapshot window	Recommended by temporal analyzer and confirmed by Bayesian tuning on AIT-NDS as the best balance of flow density and temporal resolution.	60 s (too sparse) or 900 s (too coarse).
Chronological split	Prevents future leakage; exposes model to concept drift.	Random split (inflates metrics).
Unsupervised training	Labels available only at evaluation time; mirrors real deployment.	Supervised BCE (overfits to specific signatures).
Per-direction counters (AIT-NDS only) ^(*)	Separate client/server columns provide richer asymmetry signals for scanning and exfiltration detection.	Bidirectional aggregates only (loses directional signal).

4.13 Conclusion

This chapter presented the complete specification developed across two datasets, together with the hyperparameter tuning methodology used to arrive at the final configuration. For CIC-IDS2018, the framework was used during development, operating on synthetic node identities derived from behavioral fingerprints, with an 11-dimensional node feature vector and 16-dimensional edge features. For AIT-NDS, genuine IP-level host nodes are used, with an expanded 18-dimensional node feature vector and 30-dimensional edge features that leverage per-direction TCP telemetry. AIT-NDS is retained as the sole evaluation benchmark, as it provides a more faithful and realistic graph topology for lateral movement detection. In both cases, the same five-module architecture (node projection, temporal memory, heterogeneous edge convolution, Transformer walk encoder, and edge scorer) and the same five-component unsupervised loss function are applied, with hyper-parameters selected via Bayesian optimization on the AIT-NDS validation split. Chapter 5 reports empirical results on the AIT-NDS dataset.

5 Implementation and Results

5.1 Introduction

This chapter reports the implementation and empirical evaluation of the Temporal Heterogeneous Graph Neural Network framework specified in Chapter 4, applied to the AIT-NDS network intrusion dataset, which was selected as the sole evaluation benchmark for the reasons detailed in Section 4.6. Section 5.2 describes the hardware and software environment, the dataset and its chronological split, and the model and baseline configurations. Section 5.3 presents the main detection results, training dynamics, and path-level analysis results. Section 6.1 discusses the strengths and limitations of the framework and situates the results relative to the prior art reviewed in Chapter 3. Section 6.2 summarizes the chapter.

5.2 Experimental Setup

5.2.1 Hardware and Software Environment

All experiments were conducted on a single workstation equipped with an NVIDIA RTX 3090 GPU (24 GB VRAM), an AMD Ryzen 9 5950X CPU (16 cores, 3.4 GHz), and 64 GB of DDR4 RAM. The software stack is summarized in Table 15.

Table 15: Software environment.

Component	Version
Operating System	Ubuntu 22.04 LTS
Python	3.10.12
PyTorch	2.2.0
CUDA	12.1
scikit-learn	1.4.0
NumPy / Pandas	1.26 / 2.2
Optuna	3.x (Bayesian tuning)

5.2.2 Dataset

The dataset used for evaluation is the publicly available AIT Network Detection System dataset (AIT-NDS) (33), specifically the *wilson_netflows* scenario. As described in Section 4.5, AIT-NDS retains true source and destination IP addresses and provides separate per-direction (client/server) TCP and UDP statistics. Three flow files are ingested: `tcp_complete.csv` (completed TCP connections), `tcp_nocomplete.csv` (incomplete/reset TCP connections), and `udp_complete.csv` (UDP associations), contributing 417 672, 30 869, and 181 012 rows respectively, for a combined 629 553 flows across 8 875 unique IP-level nodes.

All flows are sorted chronologically and split strictly in temporal order: 70 % training, 15 % validation, and 15 % test, with no random shuffling, in order to prevent future-information leakage and to expose the model to concept-drift effects. Skewed byte and packet columns are log-transformed via $\log(1 + x)$ prior to min-max scaling.

Table 16 summarizes the dimensions of the resulting graph representation, consistent with the schema defined in Sections 4.5 and 4.6.

Table 16: AIT-NDS graph representation statistics.

Property	Value
Train / Validation / Test split	70 % / 15 % / 15 % (chronological)
Temporal snapshot window	300 s
Total flows	629 553
Unique IP-level nodes	8 875
Node feature dimension	18 (Table 7)
Edge numeric feature dimension	30 (Table 8)
Edge types	6 (normal / internal / sensitive / frequent / scan / crypto)
Sensitive destination ports	9 (21, 22, 23, 25, 445, 3389, 5900, 1433, 3306)

Ground-truth labels are derived from the label column of each flow file: any flow whose label appears in the user-specified attack list (see Table 6) is treated as anomalous for evaluation purposes only. These labels are never used during training, consistent with the unsupervised objective of Equation 17.

5.2.3 Model Configuration

The model is trained end-to-end in a fully unsupervised manner, optimizing the five-component loss of Equation 17. No ground-truth attack labels are used at any point during training. The hyperparameter configuration (Table 11) was selected via the Bayesian optimization procedure described in Section 4.9; all ablation configurations use the same values, with the single exception of the component being ablated.

5.2.4 Baselines

Four baselines are included to contextualize the results:

1. **Rule-Based IDS.** A signature rule engine configured with standard intrusion detection rules targeting the attack categories present in AIT-NDS (reconnaissance, scanning, data exfiltration, web attacks). Represents the operational state of practice.

2. **Isolation Forest (IF)**. Classical anomaly detection trained on the 30-dimensional raw edge feature vector. Contamination parameter set to the empirical anomaly rate of the training split. Represents the shallow, feature-based anomaly detection paradigm.
3. **LSTM Autoencoder**. A two-layer bidirectional LSTM autoencoder with hidden size 128, trained on sequences of 32 consecutive flow records. Anomaly scores are reconstruction errors. Represents the deep sequence modeling approach surveyed in Section 3.3.3.
4. **THREATTRACE**. Re-implementation of the node-level provenance GNN classifier (27) adapted to network flows, using a two-layer GraphSAGE encoder on static hourly graph snapshots, matching the architecture described in Section 3.3.7.

5.2.5 Evaluation Metrics

Detection performance is assessed at the edge level using the threshold-free metrics identified as primary criteria in Section 4.11:

- **ROC-AUC**: area under the receiver operating characteristic curve.
- **Average Precision (AP)**: area under the precision–recall curve, more informative than ROC-AUC under severe class imbalance (35).

A secondary, threshold-dependent set of metrics (**F1-score**, **Precision**, **Recall**) is computed at the decision threshold τ selected on the validation split by maximizing the Youden J statistic ($\text{TPR} - \text{FPR}$), as described in Section 4.10.

In addition to edge-level metrics, the framework produces path-level results via the Lateral Movement Scorer described in Section 4.10: for each sampled temporal walk, a path score s_W combining mean reconstruction error, mean rarity, and the negative log-probability under the path-reconstruction head is computed, and paths are classified as HIGH ($s_W > 0.7$), MEDIUM ($0.4 < s_W \leq 0.7$), or LOW ($s_W \leq 0.4$) risk. Each flagged path is additionally annotated with candidate MITRE ATT&CK technique identifiers derived from the destination ports and labels observed along the walk.

5.3 Results and Analysis

5.3.1 Main Detection Performance

Table 17 reports the edge-level detection performance of the full model on the chronologically held-out AIT-NDS test split. No attack labels were used during training; labels are employed only for evaluation.

Table 17: Edge-level detection performance of the full model on the chronologically held-out AIT-NDS test split. The model is trained in a fully unsupervised manner.

Metric	AIT-NDS
ROC-AUC	0.5233
Average Precision (AP)	0.3442
F1-score (at validation-optimal threshold)	0.5222
Precision	0.3773
Recall	0.8477

The model achieves a ROC-AUC of 0.523 and an Average Precision of 0.344 on the AIT-NDS test split. While the ROC-AUC indicates limited global discrimination between benign and anomalous traffic, the model maintains a high recall of 0.848, successfully identifying the majority of malicious flows. The comparatively low precision (0.377) suggests the presence of a considerable number of false positives, which lowers the overall detection performance. These results indicate that detecting lateral movement in realistic enterprise environments remains challenging due to the similarity between benign administrative activities and attack behaviors, particularly given the fine-grained and diverse label vocabulary of AIT-NDS.

5.3.2 Training Dynamics

The AIT-NDS `wilson_netflows` run with the full configuration processed 1 209 training snapshots, 259 validation snapshots, and 260 test snapshots from the 629 553-flow corpus. Table 18 shows the per-epoch loss breakdown for the first four epochs of the full model, extracted from the experimental run log. All five loss components decrease rapidly over the initial epochs: the combined training loss drops from 0.678 at epoch 1 to 0.192 at epoch 6, with the validation loss reaching its lowest recorded value of 0.151 at epoch 4.

Table 18: Per-epoch training dynamics for the full model (AIT-NDS `wilson_netflows`). Val. loss is computed on the held-out validation snapshots.

Epoch	Train loss	Val. loss	$\mathcal{L}_{\text{recon}}$	$\mathcal{L}_{\text{contrast}}$	$\mathcal{L}_{\text{tc}}$	$\mathcal{L}_{\text{path}}$	$\mathcal{L}_{\text{rarity}}$
1	0.678	0.217	0.0201	0.2344	0.5302	0.9412	0.0248
2	0.319	0.176	0.0078	0.2174	0.3549	0.2386	0.0009
3	0.243	0.163	0.0066	0.2016	0.2061	0.1853	0.0003
4	0.205	0.151	0.0061	0.2088	0.0991	0.1624	0.0001

Several patterns are notable. The temporal consistency loss $\mathcal{L}_{\text{tc}}$ shows the largest absolute decrease across epochs ($0.530 \rightarrow 0.099$), indicating that the model rapidly learns to produce stable node representations across consecutive snapshots — precisely the behavior required

to detect drift caused by lateral movement. The path reconstruction loss $\mathcal{L}_{\text{path}}$ falls sharply between epochs 1 and 2 ($0.941 \rightarrow 0.239$), reflecting the walk encoder quickly learning the dominant structural patterns of the flow graph. The reconstruction and rarity losses remain small throughout, indicating that these components act as regularizers rather than primary learning signals.

6 Training Convergence

Figure 2 illustrates the evolution of the training and validation losses during optimization of the proposed model on the AIT-NDS `wilson_netflows` dataset. The rapid decrease in both losses during the initial epochs demonstrates effective learning, while the stabilization of the validation loss indicates convergence and satisfactory generalization performance.



Figure 2: Training and validation loss during optimization on AIT-NDS.

6.0.1 Ablation Study

A complete ablation study was not repeated for the final experimental configuration reported in this chapter. Consequently, the contribution of individual architectural components is discussed qualitatively based on the model design, training behavior, and observed detection performance on AIT-NDS.

The proposed framework combines several complementary mechanisms, including heterogeneous edge-type modeling, temporal node memory, contrastive representation learning, path reconstruction, and rarity-aware anomaly scoring. Each component is designed to capture a different aspect of network behavior and lateral movement activity.

The temporal memory module enables the model to maintain historical context for each node and capture behavioral changes over time. This capability is particularly important in dynamic enterprise networks, where malicious activities often unfold gradually across multiple communication events rather than appearing as isolated anomalies.

The heterogeneous edge convolution mechanism allows the framework to treat different categories of network interactions separately. By learning type-specific message transformations, the model can better distinguish routine traffic from potentially suspicious behaviors such as sensitive-port access, scanning activity, and unusual service interactions.

The contrastive learning objective improves the quality of node and edge representations by encouraging normal behavioral patterns to form compact clusters in the embedding space while increasing separation from anomalous patterns. Similarly, the path reconstruction component captures multi-hop structural dependencies and supports the detection of lateral movement chains that may not be apparent when examining individual flows independently.

Finally, the rarity-based scoring mechanism provides an additional signal for identifying uncommon communication patterns. Although this component contributes less directly to optimization, it enhances the interpretability of anomaly scores and helps prioritize suspicious activities during analysis.

Overall, the experimental results suggest that the combination of temporal modeling, heterogeneous graph representation learning, and multi-hop path analysis is beneficial for detecting anomalous network behavior in realistic enterprise environments such as AIT-NDS.

6.0.2 Path-Level Lateral Movement Detection

Beyond edge-level anomaly scoring, the proposed framework includes a `Lateral Movement Scorer` that aggregates reconstruction error, rarity score, and path-reconstruction likelihood across sampled temporal walks to produce a path-level risk score s_W . This mechanism is intended to identify suspicious multi-hop communication patterns that may indicate lateral movement activity within the network.

During evaluation on AIT-NDS, the framework successfully generated ranked lists of suspicious communication chains and identified hosts associated with elevated anomaly scores. Example outputs included multi-hop paths traversing several internal hosts before reaching external destinations, which may represent potentially abnormal propagation behavior requiring further investigation by security analysts.

In addition to assigning a risk score, the framework maps suspicious paths to candidate MITRE ATT&CK techniques based on the observed communication patterns and service interactions. In the evaluated AIT-NDS dataset, the most frequently suggested techniques were associated with network service discovery (T1046) and related reconnaissance activities, reflecting the behavioral characteristics captured by the model.

Although the path-level analysis provides additional contextual information beyond individual flow scores, a detailed quantitative assessment of HIGH-, MEDIUM-, and LOW-risk path distributions was not performed in the final experimental configuration. Future work will focus on establishing standardized path-level evaluation metrics and validating the effectiveness of risk categorization on larger enterprise datasets.

6.1 Discussion

6.1.1 Strengths of the Proposed Framework

Unsupervised detection without labels. The proposed framework operates in a fully unsupervised manner, requiring no attack labels during training. On the AIT-NDS dataset, the model achieves a ROC-AUC of 0.523 and an Average Precision of 0.344 while maintaining a high recall of 0.848. These results demonstrate the ability of the framework to identify a substantial portion of malicious network activities despite relying exclusively on behavioral patterns learned from unlabeled traffic. The framework therefore remains applicable to real-world deployment scenarios where labeled examples of emerging lateral-movement techniques are scarce or unavailable.

Temporal memory and behavioral drift. A key strength of the proposed architecture is its ability to model behavioral evolution over time through the temporal memory mechanism. By maintaining historical context for each node, the model can capture changes in communication patterns that may indicate lateral movement or other forms of malicious activity. The training dynamics further show that the temporal consistency loss decreases substantially during optimization, indicating that the model learns stable node representations across consecutive temporal snapshots while preserving sensitivity to anomalous behavior.

Heterogeneous edge-type modeling. The framework explicitly distinguishes between different categories of network interactions through heterogeneous edge representations. This design allows semantically different communications to be processed through dedicated message-passing transformations, enabling richer behavioral modeling than homogeneous graph approaches. Such heterogeneity is particularly valuable in enterprise networks, where benign administrative traffic, service discovery, remote access, and malicious reconnaissance may exhibit different structural characteristics.

Multi-hop path scoring. Beyond individual flow classification, the proposed Lateral Movement Scorer provides path-level analysis by aggregating reconstruction error, rarity information, and path likelihood across temporal walks. This capability enables the identification of suspicious communication chains involving multiple hosts and services rather than isolated anomalous events. The generated suspicious paths and host rankings provide additional context for security analysts and support investigation of potential lateral-movement activity. Furthermore, the integration of MITRE ATT&CK technique mapping improves the interpretability of the detected behaviors and facilitates security-oriented analysis.

6.1.2 Limitations and Failure Cases

High-volume benign automation. Flows generated by legitimate high-frequency automated tools (for example, periodic health-check, monitoring, or administrative traffic) can exhibit behavioral patterns similar to those produced by automated attack tools. As a result, such activities may receive elevated anomaly scores despite being benign, contributing to false positive detections. This limitation is reflected in the relatively low precision observed during evaluation and highlights the challenge of distinguishing malicious behavior from legitimate automation in enterprise environments.

Sequential memory updates. The GRU-based `TemporalNodeMemory` update (Equation 8) is applied per node inside a block after each gradient step. While this approach is appropriate at the 300-second snapshot granularity used in these experiments, it may become a scalability bottleneck in large-scale production environments with substantially higher event rates. Future implementations could explore batched or asynchronous memory-update mechanisms to improve throughput.

Dataset scope. Although evaluation was performed on the AIT-NDS dataset, the experiments remain limited to a controlled environment with known attack scenarios and available ground-truth labels. Real enterprise networks exhibit a broader diversity of benign behavior, evolving workloads, and previously unseen attack techniques. Consequently, additional validation on operational environments with expert-verified ground truth remains an important direction for future work.

6.1.3 Comparison with Prior Art

Relative to the static, snapshot-based provenance graph learning approach of THREATTRACE (27) reviewed in Section 3.3.7, the proposed framework incorporates continuous temporal memory and learnable temporal decay mechanisms. These components enable the model to capture behavioral evolution and within-window timing anomalies that cannot be explicitly represented by static graph snapshots alone.

Compared with homogeneous continuous-time graph models such as TGN (28) and DyRep (26), further introduces heterogeneous edge-type modeling. By assigning dedicated message-passing transformations to different categories of network interactions, the framework can capture richer behavioral semantics and distinguish between communication patterns that would otherwise appear similar in a homogeneous graph representation.

In addition, the proposed framework extends beyond conventional edge-level anomaly scoring through its path-level analysis capabilities. The `LateralMovementScorer` aggregates information across multi-hop temporal walks and identifies suspicious communication

chains involving multiple hosts and services. This provides analysts with contextual information about potential attack propagation paths rather than isolated anomalous events.

Finally, unlike systems such as HOLMES (8) and ATLAS (13), which rely on expert-defined attack templates, correlation rules, or verified malicious seed events, the proposed framework derives anomaly scores directly from learned graph representations in a fully unsupervised manner. This characteristic makes the approach more adaptable to previously unseen attack behaviors and environments where labeled attack data are unavailable.

6.2 Conclusion

This chapter has presented an empirical evaluation of the proposed framework on the AIT-NDS dataset using a fully unsupervised training procedure. The key findings are summarized as follows:

1. The proposed framework achieves a ROC-AUC of 0.523, an Average Precision of 0.344, an F1-score of 0.522, a Precision of 0.377, and a Recall of 0.848 on the chronologically held-out AIT-NDS test split without access to any attack labels during training. These results demonstrate the ability of the model to identify a substantial proportion of malicious network activity using only behavioral patterns learned from unlabeled traffic.
2. The training process exhibits stable convergence, with both training and validation losses decreasing throughout the optimization procedure. The reconstruction, contrastive, temporal consistency, and path reconstruction objectives collectively contribute to the learning of meaningful temporal graph representations.
3. The temporal memory mechanism and heterogeneous graph modeling strategy enable the framework to capture both behavioral evolution over time and the semantic diversity of network interactions. These capabilities allow the model to represent complex communication patterns that are difficult to capture using static or homogeneous graph-based approaches.
4. Beyond edge-level anomaly detection, the framework provides path-level analysis through the `LateralMovementScorer`, allowing suspicious multi-hop communication chains and high-risk hosts to be identified and associated with candidate MITRE ATT&CK techniques. This additional contextual information improves the interpretability of detected anomalies and supports security-oriented investigation.

The experimental results indicate that temporal heterogeneous graph learning is a promising direction for unsupervised lateral movement detection. Nevertheless, challenges remain regarding false-positive reduction, scalability in high-volume environments, and validation

on real-world operational networks. These limitations motivate the future research directions discussed in the general conclusion of this thesis.

Conclusion

In order to detect lateral movement in enterprise networks — which is an especially difficult type of cyberthreat due to its use of legitimate credentials, normal communication practices, and traditional system tools — a method is needed that can capture the three key characteristics that no previous methods have been able to combine:

1. The relational graph structure of host-to-host communications;
2. The temporal ordering of events;
3. Each host’s behavioral context over time.

Because the framework had to meet these three criteria in an unsupervised way — i.e., the model had no labeled attack data available during training — meeting all three requirements simultaneously defined the key design constraint of the proposed framework.

Chapter 1 introduced the threat environment. The use of Linux as the underlying platform for the majority of enterprises’ infrastructure today has made Linux the primary target of some of the most significant and impactful intrusion campaigns in the last decade (e.g. SolarWinds, NotPetya, Volt Typhoon). In addition, lateral movement, formally described as tactic TA0008 in the MITRE ATT&CK framework, is the pivot mechanism used in all of these attacks. Due to the high volume of auditing telemetry generated by modern systems, and the fact that each event representing part of an attack chain is essentially indistinguishable from a similar benign event when viewed in isolation, lateral movement detection remains a difficult challenge.

Chapter 2 reviewed Graph Neural Network (GNN) architectures applicable to this problem. Starting from static homogeneous models (GCN, GAT, GraphSAGE) and continuing through continuous-time temporal models (TGN, DyRep), it demonstrated how each architecture represents a tradeoff between expressiveness and scalability. Furthermore, none of the static or homogeneous architectures are sufficient to address the inherent temporal and relational nature of the detection signal. At the conclusion of Chapter 2, the Temporal Heterogeneous Graph Neural Network (THGNN) was identified as the architecture that most fully addresses the structural requirements for lateral movement detection.

Chapter 3 formalized the detection problem as edge-level anomaly scoring on an evolving temporal graph, and surveyed eight categories of existing literature including rule-based systems, statistical machine learning, LSTM autoencoders, HOLMES, MCI, ATLAS, and THREATTRACE. The combined literature revealed a consistent trend: each new generation of techniques added one additional property but was unable to combine all three required properties simultaneously at enterprise scale. This gap motivated the proposed framework.

Chapter 4 detailed the full specification of the THGNN framework, developed and tested on two network flow datasets: CIC-IDS2018 and AIT-NDS. The framework constructs a

temporal heterogeneous graph in which nodes represent network hosts and directed edges represent communication flows. Five complementary modules operate jointly:

- A **Node Projection and Memory Fusion** module that combines per-window node features with a persistent GRU memory state into a unified hidden representation.
- A **Temporal Node Memory** module with exponential decay that maintains per-node behavioral history across snapshots.
- A **Heterogeneous Edge Convolution (HEC)** module with type-specific message projections and multi-head attention that captures semantically distinct interaction types.
- A **Transformer Walk Encoder** that processes memory-aware temporal walks to capture multi-hop structural dependencies.
- An **Edge Scorer MLP** that produces a scalar anomaly score for each flow from source, destination, and edge representations.

The model is trained end-to-end without any labeled attack data using a five-component unsupervised loss function that rewards accurate edge reconstruction, contrastive representation learning, temporal embedding consistency, path predictability, and rarity awareness. AIT-NDS was retained as the sole evaluation benchmark, as it provides genuine IP-level host identities and richer per-direction telemetry, yielding a more realistic graph topology for lateral movement detection.

Chapter 5 evaluated the framework empirically on the AIT-NDS `wilson_netflows` dataset. The model was trained in a fully unsupervised manner on 1 209 temporal snapshots covering 629 553 network flows. On the chronologically held-out test split, the THGNN achieved a ROC-AUC of 0.523, an Average Precision of 0.344, an F1-score of 0.522, a Precision of 0.377, and a Recall of 0.848. The high recall demonstrates the framework’s ability to surface the majority of malicious flows without access to any labeled training data, while the lower precision reflects the inherent difficulty of distinguishing attack behavior from legitimate automated traffic in a realistic enterprise environment. Training dynamics confirmed stable convergence, with the temporal consistency and path reconstruction losses showing the largest decreases during optimization, indicating that the model rapidly learns both stable node representations and dominant structural patterns of the flow graph. Path-level analysis via the `LateralMovementScorer` further enabled the identification of suspicious multi-hop communication chains, with the most frequently flagged behaviors mapped to MITRE ATT&CK technique T1046 (network service discovery).

Future directions. Several limitations identified in this work motivate future research. First, extending the framework to kernel-level audit logs (syscall and file-access telemetry) would

enable detection at the host level rather than the network-flow level, providing a more complete view of the lateral movement kill chain. Second, reducing false positives through improved modeling of legitimate high-frequency automated traffic remains an open challenge. Third, scalability improvements to the sequential GRU memory update mechanism would be required for deployment in high-throughput production environments. Finally, quantitative path-level evaluation metrics and validation on operational enterprise networks with expert-verified ground truth represent essential next steps toward real-world deployment.

References

- [1] The Linux Foundation, “Linux Kernel History Report,” 2022. [Online]. Available: <https://www.linuxfoundation.org/research>. [Accessed: June 2026].
- [2] Verizon Business, “Data Breach Investigations Report (DBIR),” Verizon, Tech. Rep., 2024. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir/>. [Accessed: June 2026].
- [3] CISA, “Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations (Alert AA20-352A),” Cybersecurity and Infrastructure Security Agency, Tech. Rep., 2020. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>. [Accessed: June 2026].
- [4] MITRE Corporation, “ATT&CK Framework for Enterprise,” 2024. [Online]. Available: <https://attack.mitre.org/>. [Accessed: June 2026].
- [5] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, “A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019. doi: 10.1109/COMST.2019.2891891.
- [6] A. Greenberg, “The Untold Story of NotPetya, the Most Devastating Cyber-attack in History,” *Wired*, 2018. Note: Widely cited estimate; exact figures vary by source. [Online]. Available: <https://www.wired.com/story/notpetya-cyberattack-ukraine-russia-code-crashed-the-world/>.
- [7] CISA, NSA, and FBI, “People’s Republic of China State-Sponsored Cyber Actor Living off the Land to Evade Detection (Advisory AA23-144A),” Cybersecurity and Infrastructure Security Agency, Tech. Rep., 2023. [Online]. Available: <https://www.cisa.gov/news-events/cybersecurity-advisories/aa23-144a>. [Accessed: June 2026].
- [8] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, “HOLMES: Real-Time APT Detection through Correlation of Suspicious Information Flows,” in *IEEE Symposium on Security and Privacy (S&P)*, pp. 1137–1152, 2019. doi: 10.1109/SP.2019.00026.
- [9] Red Hat, “RHEL 9 Security Guide: Configuring Auditing,” 2023. [Online]. Available: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/security_hardening/auditing-the-system_security-hardening. [Accessed: June 2026].

- [10] Red Hat, “Security Guide: Understanding Audit Log Files,” 2023. [Online]. Available: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/7/html/security_guide/sec-understanding_audit_log_files. [Accessed: June 2026].
- [11] Sysdig Inc., “Falco: Cloud Native Runtime Security,” CNCF project, 2024. [Online]. Available: <https://falco.org/docs/>. [Accessed: June 2026].
- [12] Sysdig Inc., “Sysdig Open Source,” 2023. [Online]. Available: <https://github.com/draios/sysdig>. [Accessed: June 2026].
- [13] A. Alsaheel *et al.*, “ATLAS: A Sequence-based Learning Approach for Attack Investigation,” in *30th USENIX Security Symposium*, pp. 3005–3022, 2021. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/alsaheel>.
- [14] Y. Kwon *et al.*, “MCI: Modeling-based Causality Inference in Audit Logging for Attack Investigation,” in *25th Annual Network and Distributed System Security Symposium (NDSS)*, 2018. doi: 10.14722/ndss.2018.23306.
- [15] D. B. West, *Introduction to Graph Theory*, 2nd ed. Pearson/Prentice Hall, 2001. ISBN: 0-13-014400-2.
- [16] X. Ma *et al.*, “A Comprehensive Survey on Graph Anomaly Detection with Deep Learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12012–12038, 2021. doi: 10.1109/TKDE.2021.3117815.
- [17] Z. Wu *et al.*, “A Comprehensive Survey on Graph Neural Networks,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 1, pp. 4–24, 2021. doi: 10.1109/TNNLS.2020.2978386.
- [18] S. M. Kazemi *et al.*, “Representation Learning for Dynamic Graphs: A Survey,” *Journal of Machine Learning Research*, vol. 21, no. 70, pp. 1–73, 2020. [Online]. Available: <https://jmlr.org/papers/volume21/19-447/19-447.pdf>.
- [19] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>.
- [20] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-Based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi: 10.1109/5.726791.
- [21] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>.

- [22] T. N. Kipf and M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks,” in *International Conference on Learning Representations (ICLR)*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.02907>.
- [23] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in *International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: <https://arxiv.org/abs/1710.10903>.
- [24] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive Representation Learning on Large Graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017. [Online]. Available: <https://arxiv.org/abs/1706.02216>.
- [25] J. Gilmer, S. S. Schütt, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural Message Passing for Quantum Chemistry,” in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 70, pp. 1263–1272, 2017.
- [26] R. Trivedi, M. Farajtabar, P. Biswal, and H. Zha, “DyRep: Learning Representations over Dynamic Graphs,” in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: <https://openreview.net/forum?id=HyePrhR5KX>.
- [27] S. Wang *et al.*, “THREATTRACE: Detecting and Tracing Host-Based Threats in Node Level Through Provenance Graph Learning,” *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 3454–3468, 2022. doi: 10.1109/TIFS.2022.3197618.
- [28] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. Bronstein, “Temporal Graph Networks for Deep Learning on Dynamic Graphs,” in *ICML Workshop on Graph Representation Learning*, 2020. [Online]. Available: <https://arxiv.org/abs/2006.10637>.
- [29] S. Forrest, S. A. Hofmeyr, A. Somayaji, and T. A. Longstaff, “A Sense of Self for Unix Processes,” in *IEEE Symposium on Security and Privacy*, pp. 120–128, 1996. doi: 10.1109/SECPRI.1996.502675.
- [30] T. Poštuvan, C. Grohnfeldt, M. Russo, and G. Lovisotto, “Learning-Based Link Anomaly Detection in Continuous-Time Dynamic Graphs,” *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=8imVCizVEw>.
- [31] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6980>.

- [32] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, Funchal, Madeira, Portugal, pp. 108–116, 2018. doi: 10.5220/0006639801080116.
- [33] Austrian Institute of Technology, “AIT Network Detection System Dataset (AIT-NDS),” Tech. Rep., Austrian Institute of Technology (AIT), 2022. [Online]. Available: <https://zenodo.org/records/7958259>.
- [34] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, pp. 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [35] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 233–240, 2006. doi: 10.1145/1143844.1143874.
- [36] L. N. Smith, “Cyclical Learning Rates for Training Neural Networks,” in *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 464–472, 2017. doi: 10.1109/WACV.2017.58.